

Memoria de TFM: RouteMind Euskadi - Plataforma Analítica y Predictiva de Turismo Inteligente en el País Vasco

Trabajo Fin de Máster

Máster en Big Data & Data Engineering

Autor: Angelica Pineda Martinez

Tutores: Jorge Centeno y Alberto González

Septiembre 2026

Contents

1. Resumen.....	3
2. Palabras clave	4
3. Introducción	5
3.1. Contextualización del proyecto.....	5
3.2. Objetivos del proyecto	5
3.3 Justificación e interés.....	6
4. Metodología	8
4.1. Diseño general del proyecto.....	8
4.2. Proceso de trabajo y fases (planificación)	8
5. Arquitectura	11
5.1. Diagrama general	11
5.2. Descripción de la arquitectura técnica	11
5.3. Justificación de elección de tecnologías	12
5.4. Estimación básica de costes de infraestructura.....	13
5.5. Estrategia DevOps.....	14
6. Solución tecnológica.....	15
6.1. Fuentes de datos	15
6.2. Preparación de datos	16
6.3. Flujos de datos	17
6.4. Procesamiento.....	18
6.5. Orquestación.....	20
6.6. Almacenamiento y consulta	21
6.7. Explotación de resultados	22
7. Resultados y conclusiones	23
7.1. Logros alcanzados y validación de los resultados	23
7.2. Métricas utilizadas	23
7.3. Limitaciones identificadas	24
7.4. Futuras líneas de mejora y/o desarrollo	24
8. Anexos	27

1. Resumen

El sector turístico en el País Vasco ofrece una excepcional riqueza cultural, natural y gastronómica; sin embargo, la alta variabilidad meteorológica de la región condiciona fuertemente la experiencia de los visitantes. Ante la falta de herramientas que integren de forma dinámica las condiciones climáticas y ambientales con la oferta de ocio, surge la necesidad de optimizar la planificación de viajes mediante soluciones tecnológicas basadas en datos para evitar aglomeraciones, mitigar imprevistos climáticos y enriquecer la visita del turista.

Para dar respuesta a esta problemática, este Trabajo de Fin de Máster presenta *RouteMind Euskadi*, una plataforma analítica y predictiva de turismo inteligente. El proyecto abarca el diseño y despliegue *end-to-end* de una arquitectura de datos orientada a la generación de itinerarios personalizados para una ventana de tres meses. Su núcleo diferencial es un modelo predictivo de *Machine Learning* que analiza lecturas masivas e históricas de meteorología y calidad del aire para calcular un índice diario de viabilidad, clasificando la idoneidad de recomendar actividades *outdoor* o *indoor*.

A nivel técnico, la solución se construye sobre Microsoft Azure implementando un patrón de arquitectura Medallion. El flujo automatiza la ingesta de fuentes públicas con Azure Data Factory y procesa la información utilizando PySpark en clústeres de Azure Databricks. Los datos enriquecidos se vuelcan en una base de datos NoSQL MongoDB para alimentar una aplicación web moderna desarrollada en React y Node.js. Finalmente, la Inteligencia Artificial Generativa a través de la API de Gemini, actúa como motor conversacional para evaluar los datos filtrados y redactar planes de viaje lógicos, fluidos y completamente adaptados al perfil del usuario y al clima esperado.

2. Palabras clave

Turismo inteligente, País Vasco, Sitios Turísticos, Puntos de Interés, Itinerarios Personalizados, Meteorología, Calidad del Aire, Experiencia de Usuario, Entorno Azure, Patron de Arquitectura Medallion, Delta Lake, Azure Data Factory, Azure Databricks, MongoDB, Azure Data Lake Storage Gen 2, Modelos Predictivos, Machine Learning, MLFlow, SKLearn, Node.js, React, Gemini, Aplicación Web, Vercel.

3. Introducción

3.1. CONTEXTUALIZACIÓN DEL PROYECTO

El sector turístico en el País Vasco cuenta con una rica oferta cultural, gastronómica y natural. Sin embargo, la variabilidad meteorológica de la región juega un papel crítico en la experiencia del usuario. Actualmente, la planificación de itinerarios carece de herramientas que integren de forma automatizada y predictiva las condiciones climáticas y ambientales con la oferta de ocio. Este proyecto busca resolver dicha carencia mediante una plataforma digital basada en datos.

3.2. OBJETIVOS DEL PROYECTO

Objetivo Principal:

Diseñar e implementar una plataforma de datos escalable orientada a generar itinerarios turísticos y de ocio personalizados en el País Vasco para una ventana temporal de tres meses. Como núcleo diferencial se propone la generación de un motor de recomendación basado en Machine Learning que cruza pronósticos climáticos y de calidad del aire para obtener un índice diario de viabilidad, el cual clasificará dinámicamente la idoneidad de realizar actividades *outdoor* (al aire libre) o *indoor* (en interiores).

Objetivos Específicos:

- Construir una arquitectura Medallion (Landing, Bronze, Silver, Gold) para la ingesta, procesamiento y curado de datos masivos procedentes de datos históricos de lecturas meteorológicas de los últimos 6 años de distintos sensores posicionados en todo el territorio vasco, una API con actualización constante de eventos culturales, y listados los distintos puntos de interés como sitios de patrimonio cultural e histórico, alta gastronomía de la región y actividades de montañismo, con sus principales características.
- Desarrollar un modelo predictivo de clasificación utilizando MLflow y Python para evaluar los perfiles históricos por zonas geográficas (municipio - provincia), permitiendo generar un perfil climático diario para los próximos tres meses que determine y clasifique la viabilidad de actividades indoor/outdoor para cada día según el comportamiento estadístico consolidado.
- Integrar un sistema de almacenamiento NoSQL (MongoDB) optimizado para la capa de servicio (Serving Layer) que optimice los costes de consumo de los datos desde la interfaz de usuario.

- Desarrollar una interfaz de usuario interactiva en React que consuma los datos procesados mediante un backend en Node.js que recoja a su vez los datos personalizados de cada usuario como la zona de interés, las fechas de su visita y el tipo de actividades de preferencia, esto filtra la información previamente tratada y junto con la API de Gemini se genera un itinerario al instante de las actividades para realizar cada día.

3.3 JUSTIFICACIÓN E INTERÉS

Con el paso de los años y la masificación del turismo en el mundo, se hace cada vez más necesario realizar turismo de forma inteligente y optimizada, que evite aglomeraciones en puntos turísticos, e incluso disminución de accidentes y descongestión del tráfico en temporadas de alta afluencia.

Para el caso del País Vasco, al ser una zona geográfica con gran variedad de puntos turísticos y una importante riqueza cultural, las visitas crecen constantemente. En el informe de gasto turístico en la C.A. de Euskadi, publicado en diciembre de 2025, se resalta que el turismo en la zona representó el 6,8% del Producto Interno Bruto en 2024, creciendo un 31,1% en 5 años respecto al 2019 donde representaba un 6,3%. (Instituto Vasco de Estadística, 2025)

También es importante identificar los puntos que generan más interés entre los turistas y así entender mejor sus preferencias. Según la agencia de turismo vasca Basque Tour, en el informe Ibiltur Ocio 2025, sobre el perfil y conocimiento de los turistas que visitan Euskadi, revela que en el 2025 hubo aproximadamente 4,69 millones de turistas, de los cuales el 78% venía por vacaciones y ocio, además suelen concentrarse principalmente en Bilbao con un 29% de los visitantes y San Sebastián con un 24,5%, con actividades diversas entre las que destacan el turismo de ciudad, gastronomía, turismo costero, naturaleza y aventura, turismo rural, eventos culturales y touring.

Estas múltiples actividades han generado una progresiva desestabilización de las llegadas turísticas haciendo que los periodos prototípicamente vacacionales tan solo representen un 51% de las visitas totales durante el año. (Agencia Vasca de turismo, 2026)

Es por todo esto que se hace necesario optimizar los tiempos de visita en principales puntos de atracción turística, así como una personalización de las recomendaciones que se ajusten al perfil de cada visitante.

Para resolver esta problemática, se propone una aplicación dedicada a personalizar los itinerarios turísticos a partir de las preferencias particulares de cada usuario en

función de los cambios meteorológicos, que sería clave para reducir potencialmente las aglomeraciones en puntos principales, dando a conocer a los usuarios según sus preferencias otros puntos que pueden generar interés y enriquecer su experiencia.

La aplicación se encargará de generar itinerarios optimizados en función de métricas meteorológicas que permiten clasificar las actividades que recomienda a partir un índice que mide la viabilidad de actividades outdoor, esto debido a que el clima es un factor relevante a la hora de hacer turismo.

Las métricas meteorológicas que se tendrán en cuenta son la temperatura del aire, la precipitación, y la cantidad de partículas suspendidas en el aire de 2,5 y 10 micrómetros de diámetro, correspondiente a partículas finas que se generan en procesos de combustión de motores, o partículas gruesas como polvo y polen, respectivamente; las cuales están asociadas a enfermedades cardiovasculares y respiratorias y consideradas un riesgo ambiental para la salud a escala mundial, especialmente las partículas PM2.5 que son el contaminante con mayor carga de enfermedad atribuida. (Manglai, 2026)

Siendo el componente ambiental un valor agregado muy importante para el motor de recomendación de actividades que siempre tendrá en cuenta actividades en las cuales las condiciones climáticas sean las óptimas para el usuario final.

Para mantener una red de información sólida y actualizada es importante que la arquitectura de datos sea efectiva, que demuestre la capacidad de integrar un stack tecnológico moderno y complejo, resolviendo retos reales como la ingesta de formatos no estructurados (XML, JSON), el consumo de APIs paginadas, así como la orquestación de flujos de datos en la nube para alimentar modelos de Machine Learning en producción.

4. Metodología

4.1. DISEÑO GENERAL DEL PROYECTO

El proyecto RouteMind Euskadi sigue un diseño orientado a los datos (*Data-Driven*), estructurado bajo el paradigma de la *arquitectura Medallion* (Landing, Bronze, Silver y Gold) implementada sobre *Delta Lake* y *Microsoft Azure*. Este enfoque garantiza la escalabilidad, la trazabilidad y un procesamiento eficiente de grandes volúmenes de datos mediante un ciclo de vida iterativo.

La extracción de datos crudos se automatiza mediante Azure Data Factory hacia un Data Lakehouse. Posteriormente, se procesan los datos históricos y catálogos turísticos con PySpark en Azure Databricks. En esta fase analítica reside el núcleo inteligente del proyecto: el entrenamiento y aplicación de un modelo de *Machine Learning* que evalúa las condiciones climáticas y la calidad del aire para generar un índice de viabilidad (Indoor/Outdoor) a tres meses vista.

Para optimizar los costes operativos del entorno en la nube y garantizar respuestas de baja latencia (evitando el uso de clústeres de Databricks para consultas web), la información enriquecida con predicciones climáticas se exporta desde la capa Gold hacia una base de datos NoSQL MongoDB.

El producto final es una aplicación web interactiva desarrollada en React. Cuando un usuario parametriza su viaje (fechas, zona, intereses), el backend en Node.js consulta los mejores candidatos en MongoDB y utiliza técnicas de *Prompt Engineering* para orquestrar una petición a la API de Gemini. El Modelo de Lenguaje Grande (LLM) estructura y redacta un itinerario lógico y personalizado, ofreciendo una recomendación turística inteligente, adaptada tanto a los gustos del usuario como a las restricciones meteorológicas esperadas.

4.2. PROCESO DE TRABAJO Y FASES (PLANIFICACIÓN)

Fase de Ingesta (Capas Landing y Bronze)

Capa Landing: Contiene la configuración de pipelines en Azure Data Factory para la extracción de datos crudos de la web de OpenData Euskadi y Eustat de históricos de lecturas meteorológicas, sitios de interés y eventos culturales, los pipelines involucran linked services tipo HTTP y REST para ser usados como lectura de las fuentes de datos y distintos tipos de datasets customizados para leer datos en distintos formatos (JSON, XML, XLSX), para posteriormente ser almacenados en ADLS Gen2 usando dataset para guardar los datos en formato JSON, formato que permite

leer con facilidad información anidada, para preservar la versión de los datos, los archivos se guardan con la fecha de la ejecución de los pipelines de ADF.

Capa Bronze: Una vez los datos se encuentran dentro del data lake es posible trabajar con ellos de forma optimizada, la función de esta capa es obtener datos en formato delta para poder aplicar transformaciones de limpieza y agregación reduciendo los tiempos de trabajo en volúmenes masivos de datos, para esta fase también se añadió metadatos de trazabilidad (ingestión_timestamp, source_file), que indican cuando se procesó el archivo y el nombre del archivo fuente, información clave para la gobernanza de los datos. Por lo demás, los datos se mantienen sin transformaciones aplicadas solo se leen según el formato del archivo fuente.

Fase de Procesamiento (Capas Silver y Gold)

Capa Silver: Esta Capa contiene la lógica de desanidación de estructuras complejas encontradas dentro de los datasets, además de la aplicación de funciones que estandarizan y validan la calidad de los datos con tratamiento de valores nulos, estandarización de nombres y tratamiento de valores nulos, aplicación de tipos de datos respectivos y definición de claves para actualización de tablas mediante actualización e inserción de datos nuevos.

También se realiza una categorización para unificar los distintos tipos de sitios de interés como edificios culturales, sitios naturales y patrimonio histórico, además de la generación de una tabla maestra de estaciones meteorológicas y de calidad del aire. Esta capa es clave para realizar una depuración de los datos y obtener solo aquellos que aportan valor a cada línea de negocio.

Capa Gold: En esta capa se desarrollan notebooks de agregación para condensar la información a partir de las características tipificadas en la capa Silver, también se hace una estandarización geoespacial basada en los municipios y provincias de la región y se crea la categoría que se usará para consumir los datos en la interfaz de usuario tanto para los puntos de visita estáticos como para eventos.

Para el tratamiento de datos climáticos y de calidad del aire se realiza una agregación para estandarizar la granularidad de los datos tanto geoespacial (municipio - provincia) como temporal (diaria), las métricas usadas también se promedian buscando la misma granularidad y obtener un dataset sólido para el entrenamiento del modelo de Machine learning.

Fase de Machine Learning (Predicción climática)

Esta fase de entrenamiento y registro del modelo predictivo con MLflow haciendo uso de scikit-learn (SKlearn) realiza una evaluación de las correlaciones existentes entre las distintas variables meteorológicas y de calidad del aire históricas. El modelo aplica una clasificación tabular construyendo un perfil climático consolidado a partir de 6 años de datos para cada clúster geográfico (municipio - provincia). Posteriormente, el modelo clasifica cada día de la ventana temporal de los siguientes 3 meses determinando la idoneidad para realizar actividades outdoor o indoor en cada zona. La Capa Gold contiene, por tanto, el catálogo de eventos enriquecido con estos índices resultantes de la clasificación histórica.

Fase de Integración y Backend

Despliegue de la capa de servicio de datos (MongoDB): Los datos resultantes en la Capa Gold se exportan de forma masiva hacia una base de datos NoSQL basada en documentos MongoDB. Se utiliza un conector nativo Spark-MongoDB para realizar operaciones de actualización eficiente, y a su vez, asegurará que la capa operacional disponga de datos actualizados sin afectar la disponibilidad de la plataforma.

Desarrollo del Servicio Backend (Node.js): En esta fase se desarrolla una API REST robusta y escalable utilizando Node.js. El backend se encarga de recibir las peticiones de los usuarios, validar los parámetros de entrada (fechas del viaje, destino, preferencias del turista) y realizar consultas indexadas de baja latencia sobre las colecciones de MongoDB para extraer los mejores candidatos de ocio y turismo.

Uso de la API Gemini: Cuando el usuario solicite la generación de su itinerario, el backend en Node.js recupera de MongoDB el subconjunto de eventos y lugares óptimos previamente seleccionados por el modelo de Machine Learning. Con esta información estructurada, se construye una petición formal hacia la API de Gemini. El LLM procesa este contexto verificado y redacta de forma fluida, lógica y con lenguaje natural un itinerario turístico personalizado para el usuario.

Fase de Frontend

Se implementa una aplicación web utilizando React. La interfaz ofrece un diseño interactivo, intuitivo y adaptable (Responsive), permitiendo al usuario configurar sus filtros de viaje y visualizar los puntos de interés. Se integran mapa interactivo para permitir al usuario de tener una idea grafica de a donde será su visita, además del perfil climático que debe esperar para esa fecha.

5. Arquitectura

5.1. DIAGRAMA GENERAL

El Diagrama end-to-end de la plataforma RouteMind Euskadi contiene la arquitectura del proyecto desde la ingesta de datos desde fuentes externas hasta la implementación de la Web como producto final. (Ver Anexo 1. Diagrama end-to-end de la plataforma RouteMind Euskadi)

Diagrama de arquitectura contiene el flujo de datos detallado y los elementos principales que hacen parte de la arquitectura de datos de la solución. (Ver Anexo 2. Diagrama de arquitectura de datos de la plataforma RouteMind Euskadi)

5.2. DESCRIPCIÓN DE LA ARQUITECTURA TÉCNICA

La arquitectura se implementa bajo un enfoque nativo en la nube utilizando Microsoft Azure, estructurándose en torno al paradigma de Arquitectura Medallion (Landing, Bronze, Silver y Gold) para garantizar la calidad, trazabilidad y gobernanza de los datos a lo largo de su ciclo de vida.

El proyecto plantea varias fases, la primera se desarrolla en Azure Data Factory que actúa como orquestador principal de la extracción de las fuentes de datos e ingesta de datos crudos en Azure Data Lake Storage Gen2 (ADLS Gen2).

La segunda fase plantea el uso de Azure Databricks como motor de cómputo distribuido responsable de las transformaciones ETL/ELT que estandarizan y agregan los datos generando valor en ellos para ser usados en análisis posteriores, además del desarrollo de un modelo de Machine learning para la generación del perfil climático que evalúa la viabilidad de la realización de actividades outdoor en una ventana temporal de los siguientes 3 meses que se trabaja con librería SKLearn y se monitorizará mediante MLFlow, los datos enriquecidos con las predicciones del modelo estarán dentro de la capa Gold de la arquitectura. Los datos curados se volcarán en MongoDB, que servirá de base de datos transaccional y de lectura rápida para el Backend en Node.js.

Como última fase del proyecto se encuentra el desarrollo de la interfaz de usuario en React la cual sirve como capa de consumo, aquí se recogen datos del usuario como la fecha de su visita y sus actividades preferentes con el fin de filtrar la información y seleccionar el itinerario que mejor se adapte a cada uno, una vez se filtra el listado y se tiene el índice de actividades outdoor para las fechas concretas, se usa la API de Gemini encargada exclusivamente de organizar el itinerario personalizado.

Con esto el usuario obtendrá un es un listado de los puntos a visitar y el orden en el que se sugiere hacer el recorrido, además de la información climática que debe esperar para esos días.

5.3. JUSTIFICACIÓN DE ELECCIÓN DE TECNOLOGÍAS

La elección del conjunto de herramientas a utilizar en este proyecto se fundamenta entre otras cosas, por ser tecnologías modernas y vanguardistas en el mundo de Big Data y desarrollo Software, que permiten versatilidad en la aplicación de métodos de trabajo en distintos casos de uso. Además, el sistema estará totalmente alineado con necesidades de escalabilidad futura si se prevé incluir otras ciudades o territorios que cuenten con este tipo de información, el uso de un Data Lakehouse junto con el patrón Medallion aplicado en tecnologías Azure garantiza escalabilidad e integración.

Sumando a esto, la experiencia previa en estas tecnologías permite un mayor entendimiento del caso y la optimización de las fases del proyecto. A continuación, se explica el uso detallado de cada una.

- **Azure Data Factory:** Solución óptima y nativa para ingestas masivas y almacenamiento escalable de datos crudos en los formatos de las fuentes de datos seleccionadas (XML/JSON)
- **Azure Data Lake Storage Gen2:** La flexibilidad a la hora de almacenar cualquier tipo de dato en esta herramienta facilita el tratamiento, manejo y consulta de datos, así como la conexión con las demás herramientas utilizadas.
- **Azure Databricks (PySpark):** Permite el procesamiento distribuido en memoria, fundamental para la estandarización y clusterización de grandes volúmenes de datos geográficos y meteorológicos.
- **SKLearn:** Es la librería base para el desarrollo del modelo de ML gracias a la eficiencia que genera en tareas de clasificación regresión y clustering, garantizando una solución óptima para el motor de recomendación turística.
- **MLFlow:** Centraliza el monitoreo y cambios que tenga el modelo de ML para controlar su ciclo de vida asegurando una transición fluida y controlada desde la fase de experimentación hasta su puesta en producción.
- **MongoDB:** Su esquema flexible basado en documentos (BSON) es ideal para almacenar itinerarios complejos y jerárquicos que serán consumidos rápidamente por el frontend.
- **Node.js:** Permite la conexión a modo de backend entre el frontend y la arquitectura de datos desarrollada y es altamente integrable dentro del Framework de React.

- **API Gemini:** Al ser una API que se integra fácilmente en el entorno de desarrollo de Node.js, hace de esta una elección muy acertada. Además, esta API concreta permite hacer uso de una capa gratuita de peticiones que optimiza los costes del proyecto.
- **React:** Permite el desarrollo Frontend de manera ágil, personalizada, moderna y eficiente para el manejo de asincronía y el desarrollo de interfaces de usuario interactivas.
- **Vercel:** Plataforma de hosting que permite el despliegue de aplicaciones Web sin costo, además es altamente compatible con React como lenguaje Front, lo que lo hace realmente útil para el proyecto.

5.4. ESTIMACIÓN BÁSICA DE COSTES DE INFRAESTRUCTURA

A continuación, se muestra un resumen de los costos generados por cada recurso utilizado para el proyecto.

Recurso Utilizado	Propósito	Coste
Azure Data Factory	Ingesta de datos al sistema desde fuentes externas	€17,11
Azure Data Lake Storage Gen2	Almacenamiento de datos de la plataforma	€2,61
Azure Databricks	Preparación y Procesamiento de datos (Generación de modelo ML)	€23,89
NAT Gateway	Servicio de red que permite a los recursos del sistema conectarse a internet o redes externas	€32,53
Virtual Machines	Utilizada internamente por Azure Databricks	€5.01
Virtual Network	Utilizada internamente por Azure Databricks	€3.61
MongoDB	Capa de servicio	€0
Vercel	Despliegue de la aplicación en un Dominio publico	€0
Backend de la aplicación Web	Backend de la App Web hecho en Node.js	€0
Frontend de la aplicación Web	Frontend de la App Web realizado en React	€0
API de Gemini	Optimización de itinerarios sugeridos	€0

Esto teniendo en cuenta los volúmenes de datos usados para el proyecto, es importante aclarar que, en un entorno con mayor volumen de datos o actualización de estos, más los costes productivos de publicar la aplicación Web en un dominio propio o la utilización de la API de Gemini excediendo los límites de la capa gratuita de la API aumentarían los costes generales considerablemente.

Ver Anexo 3. Evidencia de análisis de costes de la suscripción, la cual se encuentra agrupado por los distintos recursos usados en el proyecto.

5.5. ESTRATEGIA DEVOPS

La estrategia de despliegue y control de versiones del proyecto se apoya en prácticas de Integración y Despliegue Continuo (CI/CD), asegurando agilidad y trazabilidad en el desarrollo.

El código fuente del proyecto está alojado de forma centralizada en un repositorio de GitHub. El repositorio se encuentra estructurado modularmente en dos grandes bloques para separar responsabilidades: el directorio *data_architecture/*, que contiene todo el código analítico (notebooks de PySpark, configuración de Databricks y pipelines); y el directorio *app/routemind-euskadi/*, que aloja el código fuente de la aplicación web (React y backend Node.js).

El despliegue de la aplicación web se gestiona mediante la plataforma de *hosting* Vercel. El repositorio de GitHub se encuentra enlazado y configurado en Vercel de tal forma que cualquier cambio impulsado (push) a la rama principal (master) del repositorio desencadena automáticamente un proceso de *build* y despliegue. Esto garantiza que la interfaz de usuario de RouteMind Euskadi esté siempre actualizada con las últimas iteraciones de desarrollo sin necesidad de intervención manual.

Complementario a GitHub para el código fuente, la gobernanza y versionado del modelo predictivo se gestiona nativamente dentro del entorno de Azure Databricks utilizando MLflow, lo que permite monitorizar y mantener un registro estructurado de los experimentos y el rendimiento del modelo en producción

6. Solución tecnológica

6.1. FUENTES DE DATOS

Se utilizarán las siguientes fuentes de datos públicas del portal de datos abiertos del País Vasco Open Data Euskadi y el Instituto Vasco de Estadística Eustat:

- API de eventos culturales (paginada) que contiene los eventos para todo el País Vasco para los meses de agosto, septiembre y octubre de 2026, con frecuencia de actualización diaria para actualizar eventos para estos meses que se añaden o cancelan cada día.
- Listado de sensores meteorológicos en formato JSON con fecha de actualización del 17 de agosto de 2026.
- Datos históricos de lecturas meteorológicas con frecuencia de cada 10 minutos de los últimos 6 años (2020-2025) para los meses de agosto a noviembre en formato XML.
- API de calidad del aire que contiene listado de estaciones remotas de medición y el histórico de lecturas diarias por provincia de los últimos 6 años (2020-2025) para los meses de agosto a noviembre en formato JSON.
- Información lugares gastronómicos registrados en el territorio vasco que incluyen (restaurantes, sidrerías, bodegas, bodegas de Txacolí, trujales, bares de pintxos, asadores) en formato XML con fecha de actualización del 13 de septiembre de 2026.
- Información de recursos culturales que incluyen sitios de patrimonio histórico y cultural, museos, salas de exposición, monumentos, entre otros; en formato XML, con fecha de actualización del 11 de septiembre de 2026.
- Información de edificios culturales que incluyen castillos, iglesias, edificaciones de patrimonio cultural en formato XML, con fecha de actualización del 11 de septiembre de 2026.
- Información de recursos naturales que incluyen playas, parques naturales y senderos ecoturísticos en formato XML, con fecha de actualización del 11 de septiembre de 2026.
- Listado de identificadores de municipios y provincias del País Vasco extraído de Eustat, con fecha de actualización del 14 de diciembre de 2017.

Para la ingesta de datos se utilizaron Datasets de ADF de origen HTTP o REST con formato tipo JSON, XML o Excel según el caso, esto para poder leer correctamente la estructura de los datos, luego para la almacenar los datos dentro de ADLS Gen2, se usó un dataset de destino tipo Azure Data Lake Storage Gen2 con formato JSON para unificar el tipo de datos de la capa Landing. Durante el proceso se utilizaron parámetros para dinamizar los datasets y permitir su reutilización dentro de los pipelines de ingesta.

En la sección de anexos se puede consultar el listado de linked services utilizados (Ver Anexo 4. Linked Services utilizados) y los datasets de origen y destino creados para obtener la información de la capa Landing (Ver Anexo 5. Datasets utilizados).

6.2. PREPARACIÓN DE DATOS

Respecto a los datos tomados de las distintas fuentes, es importante resaltar que los datos que incluyen temporalidad, como los eventos culturales y los históricos de lecturas meteorológicas y de calidad del aire, se ha aplicado un muestreo dirigido acotando los distintos datasets para optimizar el cómputo empleado en el procesamiento de datos. Para el caso de los eventos, se extrajeron los eventos de los siguientes tres meses ya que es la ventana temporal sobre la que se produce el pronóstico del perfil climático; para el caso de los históricos se ha acotado a los meses de interés (agosto, septiembre, octubre y noviembre) de los últimos 6 años.

También fue necesario obtener un listado oficial de identificadores de municipios y provincias del País Vasco para estandarizar los identificadores geoespaciales que usa el sistema, poder aplicar uniones entre datos usando las mismas claves y evitar incongruencias en la identificación de zonas geográficas de las distintas líneas de negocio.

La capa bronze basada en notebooks de databricks, cumple una función clave para la preparación de los datos ya que en esta fase los datos pasan a formato Delta Lake sobre ALDS Gen2 aportando las transacciones ACID lo que implica que su estructura se optimiza para permitir hacer transformaciones en los datos de forma segura e integra, además los metadatos de trazabilidad permiten siempre generar estrategias de recuperación y BackUp, mantener un versionamiento de los datos y hacer más sencilla la identificación de errores.

Para este caso se crearon tablas por cada fuente de datos, preservándolos sin transformaciones más que las necesarias para cambiar su formato a Delta y la creación de los campos de trazabilidad, se crearon notebooks por cada lógica de negocio con el fin de hacer más sencillo el flujo de los datos y su posterior orquestación.

En la sección de anexos (Ver Anexo 8. Listado de tablas Bronze) se ve el listado de las tablas generadas para esta capa y un ejemplo de la tabla (Ver Anexo 9. Ejemplo de tabla Bronze) con los campos de trazabilidad de la última actualización y la estructura con la que llega a la capa Bronze desde Landing, manteniendo los datos sin transformaciones.

Dentro de la capa Silver, también basada en notebooks para construir toda la lógica y enfocada a la limpieza de los datos, se utilizaron funciones específicas de

desanidación de Pyspark como *Explode*, con el fin de aplanar estructuras complejas de los formatos de origen (JSON/XML), también fue muy útil para resolver la falta de estandarización en los datasets de lenguaje de cadenas de texto como nombre de municipios, puesto que al estar en español y Euskera hacía que su identificación fuese compleja, para este caso también se hizo uso de otras funciones como *Split* y *Filter*, que permitían segregar la información relevante y estandarizada para el sistema.

Otras funciones importantes como *regexp_extract* permitió extraer las métricas de interés de los datos de meteorología sin generar altos costes de cómputo y usando funciones nativas.

En esta fase también se aplicaron funciones de limpieza creadas para el sistema como *explode_array_column*, *extract_and_cast*, *empty_string_to_null*, *drop_null_required*, *add_category_column* y *deduplicate* creadas para aplanar estructuras complejas, aplicar tipos de datos correspondientes, estandarización de valores nulos, eliminación de registros que no aportan valor dentro del sistema, aplicación de categorías de estandarización y eliminación de duplicados para evitar errores en la escritura de los datos dentro de las tablas objetivo.

Con esto también se realizó una limpieza de campos para obtener en las tablas resultantes de esta capa solo aquellos campos útiles para cada línea de negocio. En la sección anexos (Ver Anexo 9. Listado de tablas Silver y Anexo 10. Listado de tablas Gold) se ve el listado de las tablas generadas en esta capa junto con sus esquemas.

6.3. FLUJOS DE DATOS

El flujo es de naturaleza Batch y comienza con la ingesta de datos desde las fuentes de datos, esta primera etapa desarrollada en Azure Data Factory contempla 10 pipelines de carga de datos, uno por cada fuente de datos (lógica de negocio), además existen otros 3 pipelines auxiliares que complementan la lógica de lectura de datos en los históricos y los eventos culturales, en la sección de anexos 6 y 7 (ver Anexo 6. Listado de pipelines y Jobs del flujo de datos de la arquitectura y Anexo 7. Evidencia de ejecuciones de pipelines de ingesta en Azure Data Factory) se muestra el listado de pipelines creados y las ejecuciones generadas para ingestar los datos dentro de la capa Landing.

La lógica posterior para los flujos de datos de basa en Jobs de databricks que ejecutan los notebooks que albergan la lógica de transformación aplicada a cada caso, se organiza a nivel de capa, también se ha realizado una agrupación de los notebooks que se ejecutan dentro de cada Job con base en la frecuencia de ejecución (daily, quarterly).

Los Jobs de la capa Bronze y los de la integración de datos a MongoDB contienen una parametrización customizada para cada notebook. Para el caso de los notebooks de Bronze, el parámetro es el nombre del archivo (*file_name*), y para el caso de los notebooks de integración reciben el parámetro de frecuencia de ejecución (*load_frequency*). En la sección anexos (ver Anexo 6. Listado de pipelines y Jobs del flujo de datos de la arquitectura) se encuentra el listado de Jobs y pipelines asociados a cada capa.

Se ha definido un flujo de datos por capas para evitar inconsistencias en el procesamiento de los datos entre tablas, si hay un error dentro de la capa Bronze o Silver se puede corregir antes de realizar procesamientos en Gold con datos sin integridad entre ellos que generen resultados erróneos.

6.4. PROCESAMIENTO

El procesamiento de los datos previamente depurados y tipificados en las capas Bronze y Silver se realiza basado en Notebooks y orquestados por un clúster tipo Single Node Standard_D4s_v3 con 16GB de memoria y 4 núcleos, con Databricks Runtime tipo 15.4 LTS habilitado para Machine Learning el cual ya trae preinstalado la librería MLFlow, usada para gestionar el ciclo de vida del modelo creado para el proyecto. Esta lógica se almacena dentro de la capa Gold y tiene como objetivo generar el valor analítico y predictivo principal del proyecto.

Se estructura en dos grandes bloques de procesamiento, la agregación espacial y temporal de los datos, y el desarrollo e inferencia del modelo predictivo de Machine Learning.

1. Agregación y Estandarización Geoespacial

A partir de las tablas normalizadas de meteorología y calidad del aire de la capa Silver, se ejecutan operaciones distribuidas para crear un *dataset* sólido que sirva de base para el modelo predictivo:

- **Granularidad Temporal y Espacial:** Las lecturas meteorológicas, originalmente con una frecuencia de cada 10 minutos, se agregan a una frecuencia diaria. La información se agrupa estableciendo como clúster geográfico la combinación de municipio y provincia, cruzando los datos mediante identificadores únicos con la tabla maestra de códigos geográficos (*municipality_codes*).
- **Cálculo de Métricas Ambientales:** Se aplican funciones de agregación para calcular las medias diarias de temperatura, medias de precipitación y el

promedio diario de la concentración de partículas en suspensión (PM2.5 y PM10).

- **Tipificación de Puntos de Interés y Eventos:** Para las tablas de eventos culturales y lugares turísticos estáticos (gastronomía, recursos naturales, patrimonio), se genera una categorización consolidada (`events_user_category`, `visit_points_user_category`). Esto garantiza que los datos estén perfectamente estructurados para ser volcados en la base de datos NoSQL e indexados eficientemente desde la aplicación web.

2. Modelo Predictivo de Viabilidad Climática (Machine Learning) El núcleo diferenciador del proyecto recae en el modelo de Machine Learning encargado de pronosticar el perfil climático de los próximos 3 meses.

- **Entrenamiento y Clasificación:** Se utiliza la librería *Scikit-Learn (SKLearn)* en Python para entrenar un modelo de clasificación a partir del histórico de 6 años de datos meteorológicos y de calidad del aire. El modelo evalúa las correlaciones multivariantes de los clústeres geográficos para emitir un índice diario. Ver sección de anexos (Ver Anexo 11. Evidencia del modelo de ML resultante con sus métricas.) con las características del modelo resultante.
- **Scoring de Actividades (Indoor / Outdoor):** El algoritmo genera una inferencia sobre la ventana temporal objetivo (3 meses vista). A cada día y para cada municipio, se le asigna un índice de viabilidad que clasifica la jornada como apta para actividades al aire libre (Outdoor) o si, por el contrario, las condiciones climatológicas exigen recomendar actividades de interior (Indoor).
- **Gobernanza con MLflow:** Todo el ciclo de vida del modelo de Machine Learning es gestionado a través de *MLflow*. Los hiperparámetros, las métricas de rendimiento y los artefactos del modelo quedan registrados en cada ejecución (*run*), asegurando la reproducibilidad y trazabilidad en el entorno de Azure.
- **Exportación de Resultados:** Las predicciones finales enriquecen los datos curados y se almacenan en formato Delta en la tabla `weather_prediction_scoring`. Con esto, los datos quedan listos en la capa Gold para su exportación masiva y optimizada hacia la capa de servicio en MongoDB.

Las funciones utilizadas dentro del proyecto para limpieza de datos fueron testeadas usando la librería de Python *Pytest*, la ejecución de los test unitarios se realizaron en un entorno local creando un entorno virtual con `pyspark==3.5.1`, y ejecutando el

fichero `data_architecture\tests\test_cleaning_functions.py` que contiene todos los tests unitarios, en los anexos (Ver Anexo 15. Ejecución exitosa de test unitarios) se evidencia la exitosa ejecución de los tests.

6.5. ORQUESTACIÓN

En Azure Data Factory se controla la orquestación inicial de la ingesta de datos medio de triggers aplicados a los pipelines que automatizan las cargas de datos en cada caso según su frecuencia de actualización, dentro del sistema hay dos triggers para ejecutar todos los pipelines, *Quarterly trigger*, ejecuta el pipeline el primer día del mes de cada 3 meses a la misma hora (12:00), usado para datos estáticos con muy baja tendencia a cambiar, como datos de estaciones de medición meteorológica, listado de restaurantes y bodegas y sitios culturales y naturales; *Daily trigger* ejecuta todos los días a la misma hora (12:00) el pipeline, solo es utilizado por el pipeline de eventos culturales que necesita actualizarse diariamente.

Para el caso de los datos históricos no se ha generado un plan de orquestación ya que en los propósitos del presente proyecto es una carga única tipo One Time Load que trae los datos necesarios para generar el pronóstico objetivo del proyecto, dentro de la sección 7.5 Futuras líneas de mejora y Desarrollo se propone un plan de orquestación para históricos.

La orquestación de los datos de las siguientes capas (Bronze, Silver, Gold que incluye la integración a MongoDB) se hace por medio triggers aplicados a de Jobs de Databricks para las ejecuciones diarias y trimestrales, en este caso existen un trigger para cada tipo de job que se encarga de ejecutar los datos con una ventana de tiempo considerable entre la carga de datos a de una capa a otra para evitar que las ejecuciones entre capas se solapen.

Los triggers para cada capa son los siguientes:

- **Bronze**
 - Trigger diario asociado al Job *Bronze_daily_loading*, se ejecuta a las 13:00 h.
 - Trigger trimestral asociado al Job *Bronze_quarterly_loading*, se ejecuta el día 1 del mes de cada 3 meses a las 13:00 h.
- **Silver**
 - Trigger diario asociado al Job *Silver_daily_loading*, se ejecuta a las 14:00 h.
 - Trigger trimestral asociado al Job *Silver_quarterly_loading*, se ejecuta el día 1 del mes de cada 3 meses a las 14:00 h.
- **Gold**

- Trigger diario asociado al Job *Gold_daily_loading*, se ejecuta a las 15:00 h.
- Trigger trimestral asociado al Job *Gold_quarterly_loading*, se ejecuta el día 1 del mes de cada 3 meses a las 15:00 h.
- Trigger diario asociado al Job *Daily_load_Gold_to_MongoDB*, se ejecuta a las 15:30 h.
- Trigger trimestral asociado al Job *Quarterly_load_Gold_to_MongoDB*, se ejecuta el día 1 del mes de cada 3 meses a las 15:30 h.

En la sección de anexos (Ver Anexo 12. Jobs de Databricks con triggers asociados) se encuentran los triggers asociados a cada flujo de trabajo.

6.6. ALMACENAMIENTO Y CONSULTA

Para garantizar el equilibrio entre el procesamiento analítico de grandes volúmenes de datos y la necesidad de respuestas de baja latencia en la aplicación web, el proyecto implementa una estrategia de almacenamiento híbrido y desacoplado, separando claramente el entorno analítico del operacional

- **Data Lake (ADLS Gen2):** Actúa como la fuente única de la arquitectura analítica. Alberga las capas Landing, Bronze, Silver y Gold. Gracias al formato de almacenamiento estructurado *Delta Lake*, se garantizan transacciones ACID, control de versiones (Time Travel) y esquemas fuertemente tipados. Este entorno es inmutable desde el punto de vista operacional y está dedicado exclusivamente a las transformaciones masivas (Batch) y al entrenamiento del modelo predictivo en Databricks.
- **MongoDB:** Integrada como Serving Layer para evitar los altos costes de mantener clústeres de Databricks encendidos permanentemente para servir peticiones web. Al ser una base de datos NoSQL orientada a documentos resulta idónea para almacenar las estructuras anidadas de los eventos y puntos turísticos de la capa Gold. La integración de los datos con esta capa se realiza mediante el uso del conector nativo Spark-MongoDB, configurado en el notebook *gold_to_mongoDB* que exporta masivamente las tablas enriquecidas de la capa Gold hacia colecciones de MongoDB *visit_points_user_category*, *weather_prediction_scoring* y *events_user_category*. Ver sección en anexos (Ver Anexo 13. Evidencia de base de datos MongoDB con colecciones creadas) de las colecciones creadas.

Las escrituras hacia MongoDB se realizan mediante operaciones de actualización e inserción basadas en las claves primarias de los registros, permitiendo que el catálogo de recomendaciones y predicciones climáticas actualizado diariamente.

La estructura en colecciones permite al backend realizar búsquedas indexadas de alta velocidad para filtrar actividades según las fechas y las preferencias seleccionadas por el usuario.

6.7. EXPLOTACIÓN DE RESULTADOS

Como producto final de la solución se ha creado una aplicación web llamada RouteMind Euskadi. La interfaz de usuario de la aplicación ha sido construida como una *Single Page Application* (SPA) utilizando *React* apoyado en el empaquetador *Vite* para optimizar el rendimiento. El diseño es responsive y se ha utilizado *TailwindCSS*, incorporando animaciones fluidas mediante *Framer Motion*.

El usuario interactúa con un formulario intuitivo donde ingresa los parámetros de su viaje (fechas en la ventana de los próximos 3 meses, zona geográfica de interés y categorías de ocio preferidas). Dentro de la interfaz existen distintas opciones geográficas que el usuario puede seleccionar para definir donde quiere realizar su visita, el mapa interactivo permite ubicar con mayor facilidad donde estará centralizada su visita. Una vez se genera el itinerario, el listado de opciones a realizar en cada momento del día se despliega junto con recomendaciones para su visita y el perfil climático esperado para cada jornada.

El servidor intermedio está desarrollado en *Node.js* y actúa como puente entre la capa de servicio (MongoDB) y la interfaz de usuario, gestionando la lógica de negocio y la generación inteligente de itinerarios. Cuando el backend recibe los parámetros de la solicitud desde el frontend, consulta directamente MongoDB para extraer los eventos, lugares gastronómicos o recursos naturales que coinciden con los filtros del usuario. Crucialmente, extrae también el *scoring* diario de viabilidad (Indoor/Outdoor) calculado por el modelo de Machine Learning para esas fechas y ubicaciones concretas. Luego emplea técnicas de *Prompt Engineering*. Construye un *prompt* estructurado inyectando los datos recuperados de MongoDB como contexto base y realiza una petición a la *API de Gemini (LLM)*.

El modelo de lenguaje (Gemini) procesa este contexto verificado y redacta un itinerario cohesivo, lógico y con lenguaje natural. Estructura el plan de viaje día a día, recomendando dinámicamente actividades al aire libre o en recintos cerrados basándose estrictamente en las predicciones climáticas aportadas y el catálogo de eventos del sistema.

El proceso de publicación de la aplicación RouteMind Euskadi se ha hecho usando un dominio de Vercel público al cual se accede desde el enlace <https://routemind-euskadi.vercel.app/>, donde los usuarios pueden crear sus itinerarios. (Ver Anexo 14. Evidencia de aplicación Web RouteMind Euskadi.)

7. Resultados y conclusiones

7.1. LOGROS ALCANZADOS Y VALIDACIÓN DE LOS RESULTADOS

El desarrollo de RouteMind ha culminado con éxito en la implementación de una plataforma analítica y predictiva *end-to-end*, validando la viabilidad de integrar arquitecturas de *Big Data* con modelos de *Inteligencia Artificial Generativa* y *Machine Learning*.

Se ha desplegado exitosamente una arquitectura Medallion sobre Microsoft Azure, logrando automatizar la ingesta de fuentes heterogéneas (JSON, XML) mediante Azure Data Factory y su curado progresivo con PySpark en Azure Databricks.

Se ha entrenado un modelo de clasificación multivariante que genera un perfil climático dinámico, logrando traducir variables meteorológicas y de calidad del aire en un índice práctico de idoneidad (Indoor/Outdoor) para actividades turísticas.

La exportación del catálogo curado hacia MongoDB ha demostrado ser una solución óptima para reducir la latencia de consulta y minimizar los costes computacionales en la nube, habilitando una capa de servicio altamente disponible.

La integración del *backend* (*Node.js*) con la API de *Gemini* ha validado la capacidad técnica de utilizar un LLM para orquestar datos estructurados (fechas, ubicaciones, scoring climático) y transformarlos en planes de viaje lógicos, personalizados y redactados en lenguaje natural.

7.2. MÉTRICAS UTILIZADAS

En cuanto a la elección de métricas meteorológicas y de calidad del aire se seleccionaron la temperatura del aire, la precipitación y partículas en suspensión PM10 y PM2.5 útiles para una predicción óptima de un clima agradable y sano para realizar actividades al aire libre.

Métricas del Modelo de Machine Learning (Clasificación): Al tratarse de un modelo que predice la viabilidad basada en históricos, se ha evaluado mediante las métricas estándar de clasificación registradas en MLflow:

Accuracy: Logrando un 88.6% de acierto general en la predicción del perfil climático diario frente a los datos de prueba.

F1-Score: Garantiza un equilibrio entre la precisión y la exhaustividad del modelo, obteniendo un valor de 0.9382.

En cuanto al tiempo de respuesta de la aplicación Web, desde que el usuario realiza la petición hasta que se devuelve los detalles del itinerario generado, oscila entre los 5 y 30 segundos dependiendo de los criterios de búsqueda.

7.3. LIMITACIONES IDENTIFICADAS

Durante el desarrollo del proyecto se identificaron las siguientes limitaciones técnicas:

Límite de costes en el cómputo del procesamiento e ingesta: Debido a la limitación económica que presenta el proyecto se ajustaron mínimos básicos en cuanto al cómputo y tiempos de uso, así como el volumen de datos a procesar, se decidió acotar el volumen de datos inicial para los históricos para evitar sobrecostes en el procesamiento de un volumen mayor al usado dentro del proyecto, además la carga de históricos funciona en el proyecto como una carga única *One Time Load*, ya que actualizar el volumen de datos con una frecuencia mayor, generaría grandes costes de cómputo, sobre todo en la ingesta con Azure Data Factory que suelen ser los costes más elevados.

Los tiempos de uso del cómputo se limitaron a 10 minutos en tiempo de inactividad para evitar costes añadidos por el cómputo activo sin uso permanente.

Dependencia de APIs de terceros: la arquitectura de ingesta depende estrictamente de la disponibilidad e inmutabilidad de la estructura de las fuentes de datos. Un cambio no identificado puede causar errores de lectura o en transformaciones posteriores de los datos.

Límites de Cuota en la Generación de IA: El uso de la capa gratuita de la API de Gemini para la redacción de itinerarios impone restricciones en la cantidad de peticiones por minuto y por día.

7.4. FUTURAS LÍNEAS DE MEJORA Y/O DESARROLLO

A partir de la arquitectura propuesta se propone desarrollar pipelines incrementales que actualicen estas lecturas mensualmente para implementar un reentrenamiento automático del modelo en *MLflow*, evitando la degradación de este.

En cuanto a los tipos de datos utilizados para recomendar los itinerarios, una variable muy importante que podría añadirse es el tráfico vehicular en la zona, actualmente la web de Open Data Euskadi cuenta con una API de Tráfico que ofrece datos sobre cámaras de tráfico, incidencias y densidad de tráfico, información que aporta mucho valor a la planeación de itinerarios en la región.

Como un punto de evolución de la App Web se propone la implementación de un sistema de autenticación de usuarios en la aplicación, esto permitiría a los usuarios guardar sus itinerarios y valorarlos, así como un archivo exportable tipo PDF con el itinerario generado para que el usuario pueda consultarlo aun estando fuera de la aplicación.

Por último, es importante mencionar que debido a la alta escalabilidad que tiene la plataforma, es totalmente posible añadir información para otras zonas de España o del mundo, siempre que se tenga la misma disponibilidad de datos, es totalmente adaptable a cualquier región, tanto la arquitectura como el modelo predictivo desarrollado.

Referencias

1. El gasto turístico en la C.A. de Euskadi aumentó un 8,6 % y alcanzó los 6.665 millones de euros. Instituto Vasco de Estadística. Nota de prensa de 26/12/2025. Tomado de:
https://es.eustat.eus/estad/id_140/ti_cuenta-satelite-del-turismo/ultima-nota-prensa.html
2. Conocimiento del perfil y comportamiento de las y los turistas que visitan Euskadi por motivaciones de ocio. IBILTUR Ocio 2025. Agencia Vasca de turismo. Marzo 2026. Tomado de:
https://www.euskadi.eus/contenidos/documentacion/obs_tur_est_inf_dest_i_bilturoc/es_def/adjuntos/2025_IBILTUR_OCIO_Informe-General.pdf
3. Partículas PM2.5 y PM10: qué son, límites y calidad del aire. Manglai, 1 de julio de 2026. Tomado de:
<https://www.manglai.io/blog/particulas-pm25-pm10-calidad-aire#qu%C3%A9-son-las-part%C3%ADculas-en-suspensi%C3%B3n>
4. API de eventos culturales. Tomada de:
https://opendata.euskadi.eus/api-culture/?api=culture_events
5. Listado de sensores meteorológicos. Tomado de:
<https://opendata.euskadi.eus/catalogo/-/red-de-estaciones-meteorologicas-de-euskadi/>
6. Datos históricos de lecturas meteorológicas, Tomado de:
<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2026/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2025/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2024/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2023/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2022/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2021/>

<https://opendata.euskadi.eus/catalogo/-/estaciones-meteorologicas-lecturas-recogidas-en-2020/>

7. API de calidad del aire. Tomada de:

<https://opendata.euskadi.eus/api-air-quality/?api=air-quality>

8. Información lugares gastronómicos. Tomado de:

https://opendata.euskadi.eus/contenidos/ds_recursos_turisticos/restaurants_sidrerias_bodegas/opendata/restaurants.xml

9. Información de recursos culturales. Tomado de:

https://opendata.euskadi.eus/contenidos/ds_recursos_turisticos/patrimonio_recursos_culturales/opendata/recursos-culturales.xml

10. Información de edificios culturales. Tomado de:

https://opendata.euskadi.eus/contenidos/ds_recursos_turisticos/edificios_rel_igiosos_castillos/opendata/edificios.xml

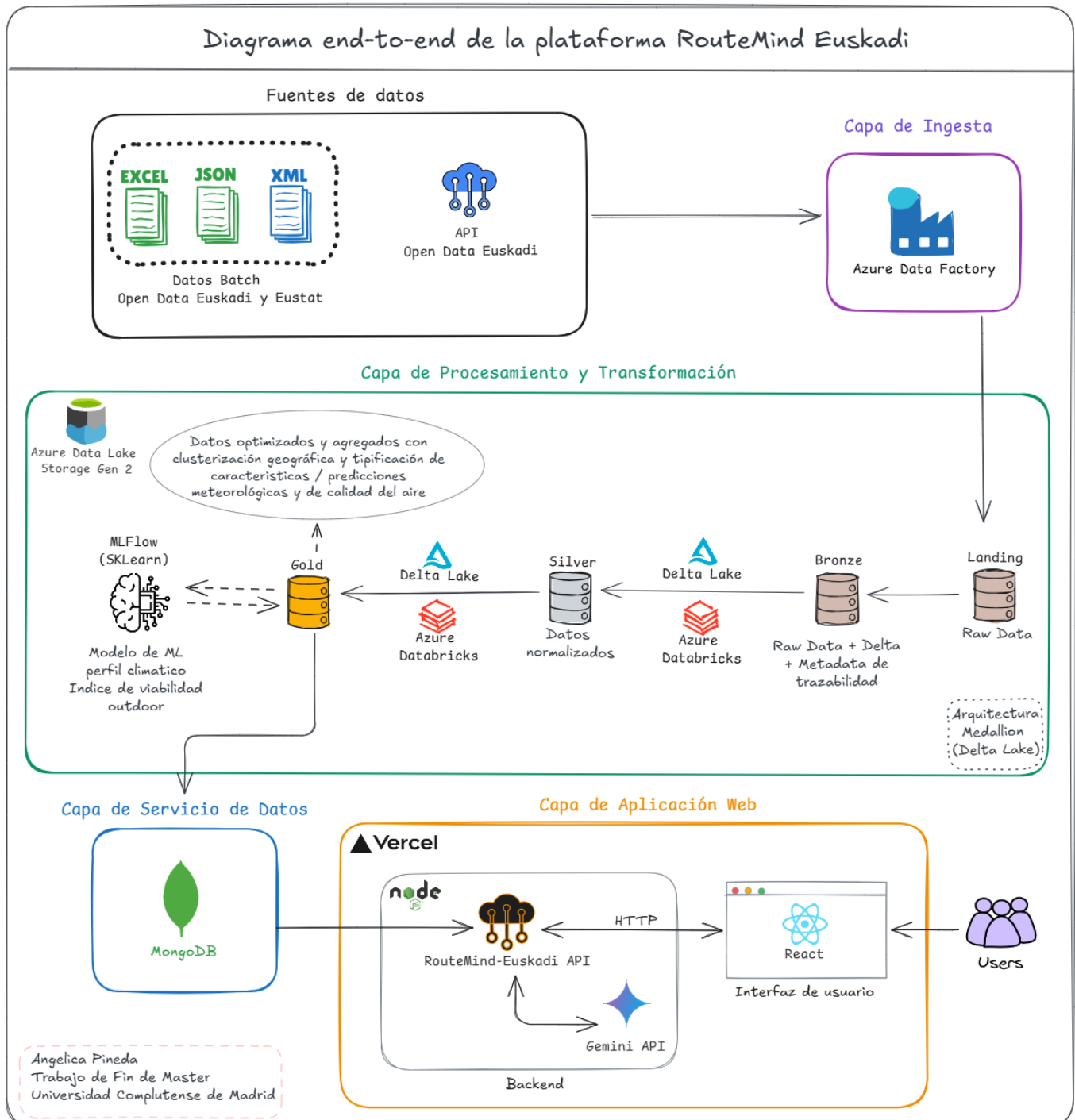
11. Información de recursos naturales. Tomado de:

https://opendata.euskadi.eus/contenidos/ds_recursos_turisticos/espacios_naturales_euskadi/opendata/espacios-naturales.xml

12. Listado de identificadores de municipios y provincias del País Vasco. Tomado de:

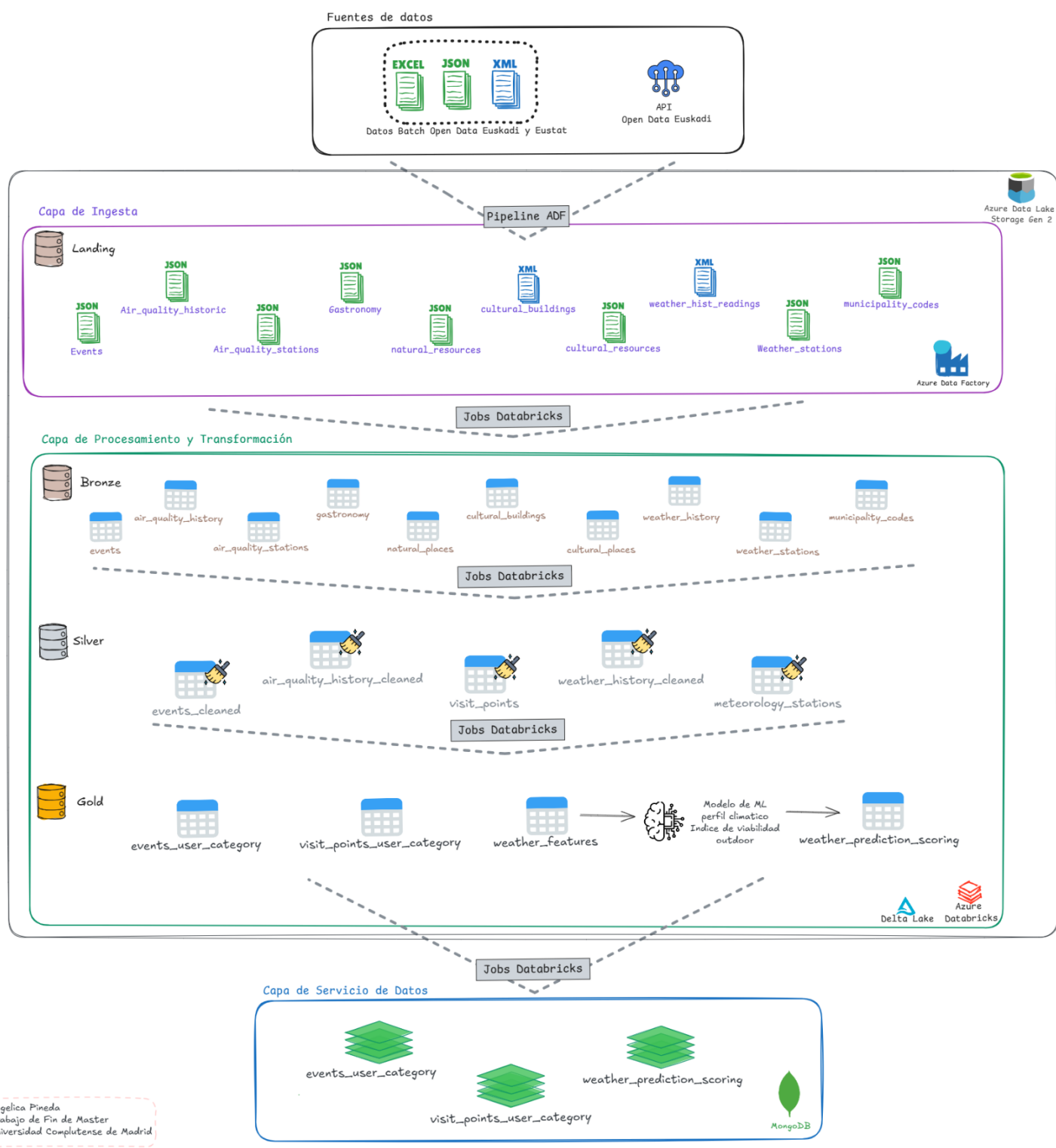
https://es.eustat.eus/estadisticas/opt_7/id_22070/clasificaciones.html

8. Anexos



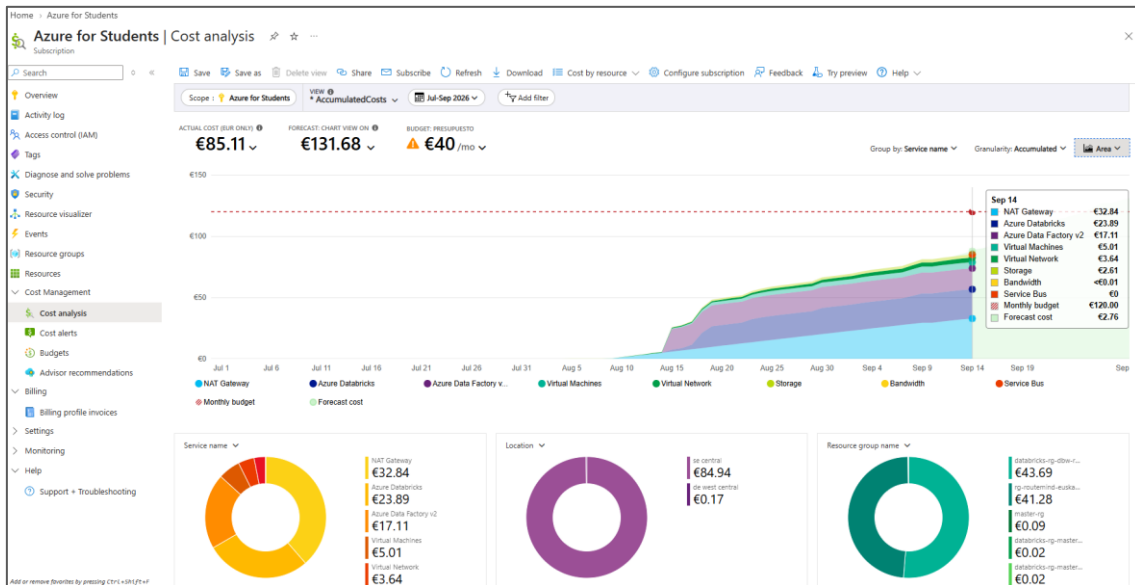
Anexo 1. Diagrama end-to-end de la plataforma RouteMind Euskadi.

Diagrama de Arquitectura de Datos RouteMind Euskadi

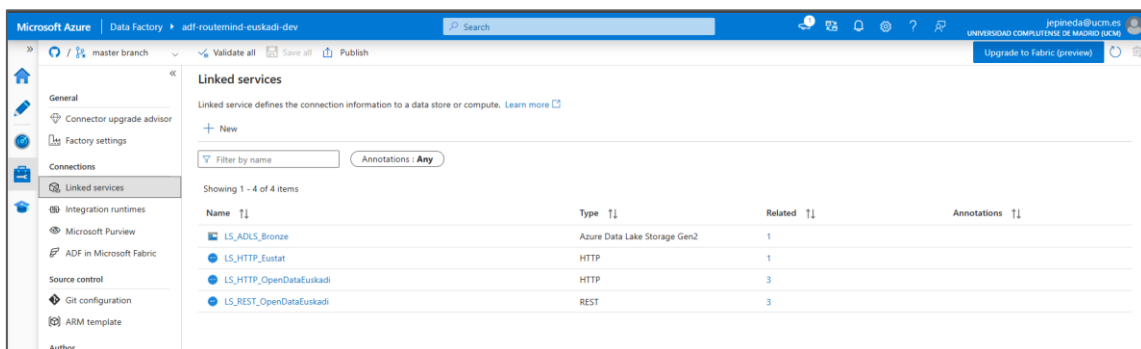


Angelica Pineda
 Trabajo de Fin de Master
 Universidad Complutense de Madrid

Anexo 2. Diagrama de arquitectura de datos de la plataforma RouteMind Euskadi.



Anexo 3. Evidencia de análisis de costes de la suscripción



Microsoft Azure | Data factory | adf-routemind-euskadi-dev

Validate all | Save all | Publish

Linked services

Linked service defines the connection information to a data store or compute. [Learn more](#)

+ New

Filter by name | Annotations: Any

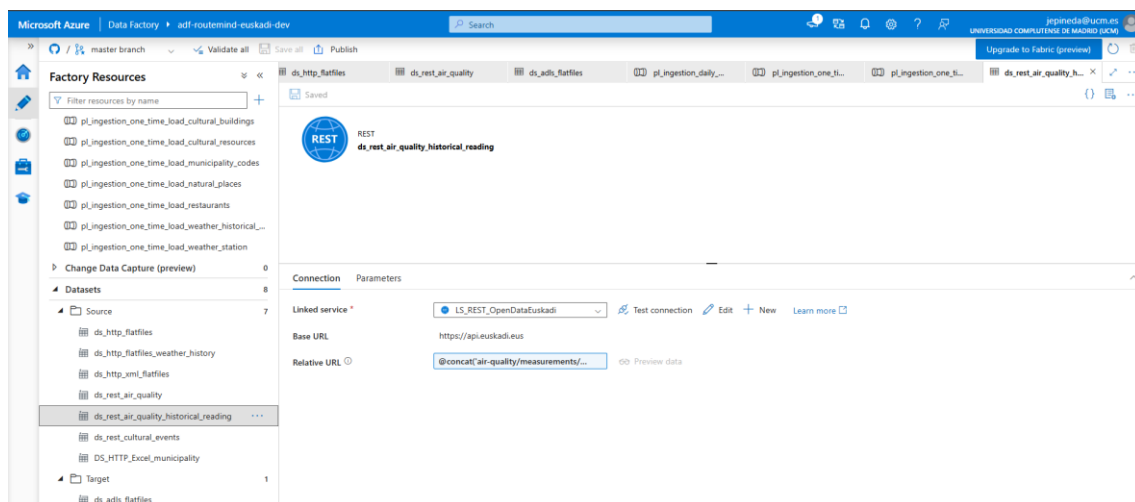
Showing 1 - 4 of 4 items

Name	Type	Related	Annotations
LS_ADLs_Bronze	Azure Data Lake Storage Gen2	1	
LS_HTTP_Estat	HTTP	1	
LS_HTTP_OpenDataEuskadi	HTTP	3	
LS_REST_OpenDataEuskadi	REST	3	

Anexo 4. Linked Services utilizados

▲ Datasets	8
▲ Source	7
ds_http_flatfiles	
ds_http_flatfiles_weather_history	
ds_http_xml_flatfiles	
ds_rest_air_quality	
ds_rest_air_quality_historical_reading	
ds_rest_cultural_events	
DS_HTTP_Excel_municipality	
▲ Target	1
ds_adls_flatfiles	

Anexo 5. Datasets utilizados



The screenshot shows the Microsoft Azure Data Factory interface. On the left, the 'Factory Resources' pane lists various datasets under 'Source' and 'Target'. The 'Source' folder contains 7 datasets, including 'ds_rest_air_quality_historical_reading'. The main pane displays the configuration for this dataset, which is a REST dataset. The 'Connection' tab is active, showing the 'Linked service' as 'LS_REST_OpenDataEuskadi'. The 'Base URL' is 'https://api.euskadi.eus' and the 'Relative URL' is '@concat("air-quality/measurements/...")'. The 'Parameters' tab is also visible.

Anexo 5.1. Ejemplo de parametrización de dataset

Anexo 6. Listado de pipelines y Jobs del flujo de datos de la arquitectura

- Landing**

Nombre del pipeline	Fuente de datos
pl_ingestion_one_time_load_weather_station	Estaciones meteorológicas
pl_ingestion_one_time_load_weather_historical_readings	Históricos de meteorología
pl_ingestion_one_time_load_restaurants	Restaurantes, etc.
pl_ingestion_one_time_load_natural_places	Recursos naturales
pl_ingestion_one_time_load_municipality_codes	Identificadores de municipios y Provincias
pl_ingestion_one_time_load_cultural_resources	Recursos Culturales
pl_ingestion_one_time_load_cultural_buildings	Edificios Culturales
pl_ingestion_one_time_load_air_quality_station	Estaciones de Calidad del aire
pl_ingestion_one_time_load_air_quality_historical_readings	Históricos de Calidad del aire
pl_ingestion_daily_api_cultural_events	Eventos Culturales

- Bronze**

Nombre del Job	Tablas
Bronze_daily_loading	events
Bronze_quarterly_loading	cultural_buildings cultural_places gastronomy natural_places weather_stations air_quality_stations

- Silver**

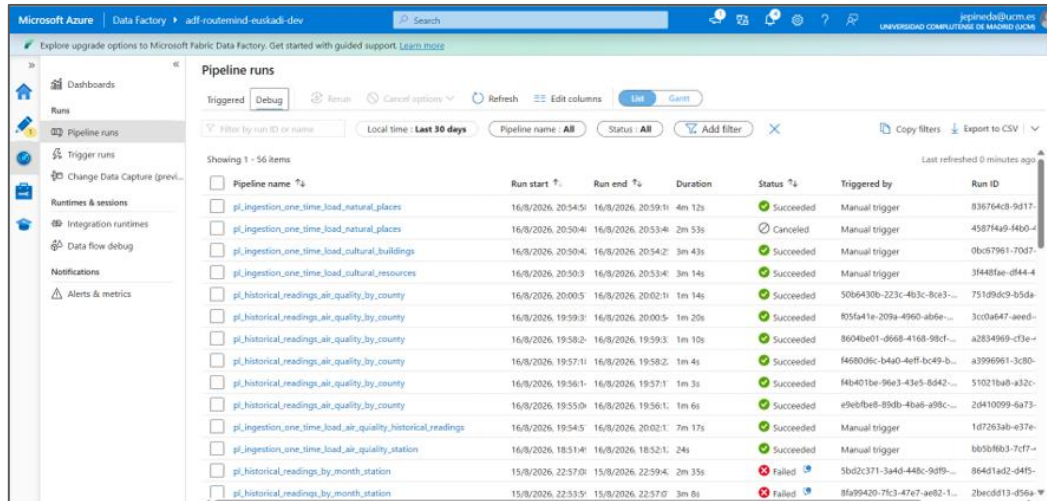
Nombre del Job	Tablas
Silver_daily_loading	Events_cleaned
Silver_quarterly_loading	visit_points meteorology_stations

- Gold**

Nombre del Job	Tablas
Gold_daily_loading	weather_prediction_scoring events_user_category
Gold_quarterly_loading	visit_points_user_category

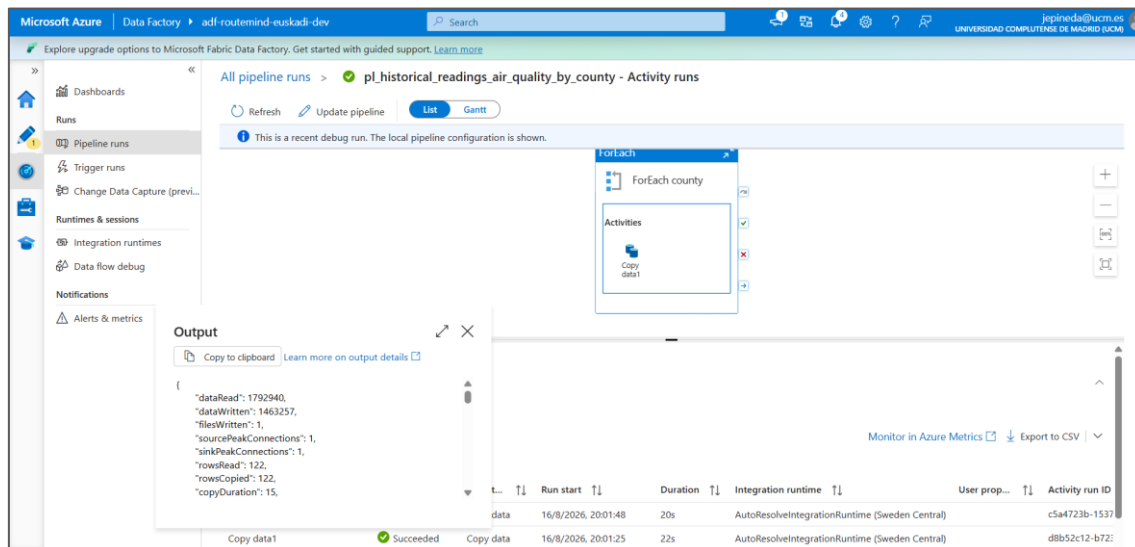
Daily_load_Gold_to_MongoDB	Integración a MongoDB
Quarterly_load_Gold_to_MongoDB	Integración a MongoDB

Anexo 7. Evidencia de ejecuciones de pipelines de ingesta en Azure Data Factory



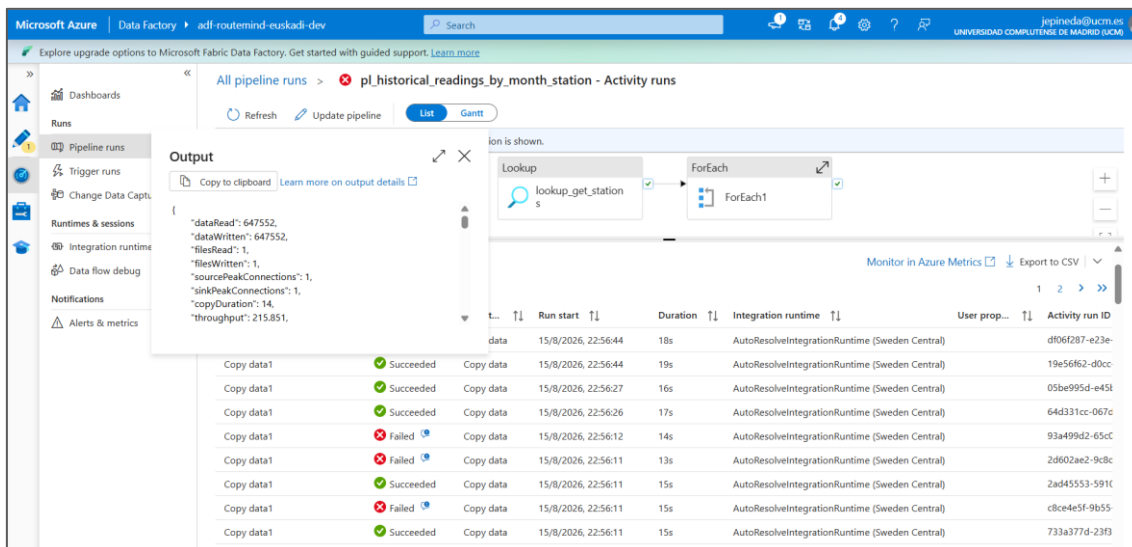
Pipeline name	Run start	Run end	Duration	Status	Triggered by	Run ID
pl_ingestion_one_time_load_natural_places	16/8/2026, 20:54:51	16/8/2026, 20:59:11	4m 12s	Succeeded	Manual trigger	836764c9-9d17-
pl_ingestion_one_time_load_natural_places	16/8/2026, 20:50:41	16/8/2026, 20:53:41	2m 53s	Canceled	Manual trigger	458774a9-14b0-4
pl_ingestion_one_time_load_natural_places	16/8/2026, 20:50:41	16/8/2026, 20:54:21	3m 43s	Succeeded	Manual trigger	0bc57991-70d7-
pl_ingestion_one_time_load_natural_places	16/8/2026, 20:50:31	16/8/2026, 20:53:41	3m 14s	Succeeded	Manual trigger	3f448f6e-df44-4
pl_historical_readings_air_quality_by_county	16/8/2026, 20:00:51	16/8/2026, 20:02:11	1m 14s	Succeeded	50b6430b-223c-4b3c-8ca3-...	75109dc9-b5da-
pl_historical_readings_air_quality_by_county	16/8/2026, 19:59:31	16/8/2026, 20:00:51	1m 20s	Succeeded	805fa41e-209a-4960-ab6a-...	3cc0a647-aeed-
pl_historical_readings_air_quality_by_county	16/8/2026, 19:58:21	16/8/2026, 19:59:31	1m 10s	Succeeded	8604be01-d668-4168-98c1-...	a2834969-c33e-
pl_historical_readings_air_quality_by_county	16/8/2026, 19:57:11	16/8/2026, 19:58:21	1m 4s	Succeeded	f468009c-b4d0-4eff-bc49-b...	a3996961-3c90-
pl_historical_readings_air_quality_by_county	16/8/2026, 19:56:11	16/8/2026, 19:57:11	1m 3s	Succeeded	f4b401be-96e3-43e3-6442-...	51021ba8-a32c-
pl_historical_readings_air_quality_by_county	16/8/2026, 19:55:01	16/8/2026, 19:56:11	1m 6s	Succeeded	e9ebf8e8-89db-4ba6-a98c-...	2d410099-6a73-
pl_ingestion_one_time_load_air_quality_historical_readings	16/8/2026, 19:54:51	16/8/2026, 20:02:11	7m 17s	Succeeded	Manual trigger	1d7263ab-e37e-
pl_ingestion_one_time_load_air_quality_station	16/8/2026, 18:51:41	16/8/2026, 18:52:11	24s	Succeeded	Manual trigger	bb5b8b03-7c7f-
pl_historical_readings_by_month_station	15/8/2026, 22:57:01	15/8/2026, 22:59:41	2m 35s	Failed	5bd2c371-3a4d-448c-9d9f-...	864d1ad2-d4f5-
pl_historical_readings_by_month_station	15/8/2026, 22:53:51	15/8/2026, 22:57:01	3m 8s	Failed	8fa99420-71c3-47e7-ae82-1...	2bcedf13-d56a-

Anexo 7.1. Evidencia de vista Monitor con las ejecuciones realizadas



Activity	Run start	Duration	Integration runtime	User prop...	Activity run ID
Copy data1	16/8/2026, 20:01:48	20s	AutoResolveIntegrationRuntime (Sweden Central)		c5a4723b-1537-
Copy data	16/8/2026, 20:01:25	22s	AutoResolveIntegrationRuntime (Sweden Central)		dbb52c12-b722-

Anexo 7.2. Evidencia de ingesta de datos históricos de calidad del aire



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Dashboard | Pipeline runs | Trigger runs | Change Data Capture | Runtimes & sessions | Integration runtime | Data flow debug | Notifications | Alerts & metrics

All pipeline runs > pl_historical_readings_by_month_station - Activity runs

Refresh Update pipeline List Gantt

Output

Copy to clipboard Learn more on output details

```

{
  "dataRead": 647552,
  "dataWritten": 647552,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "copyDuration": 14,
  "throughput": 215.851,
}
    
```

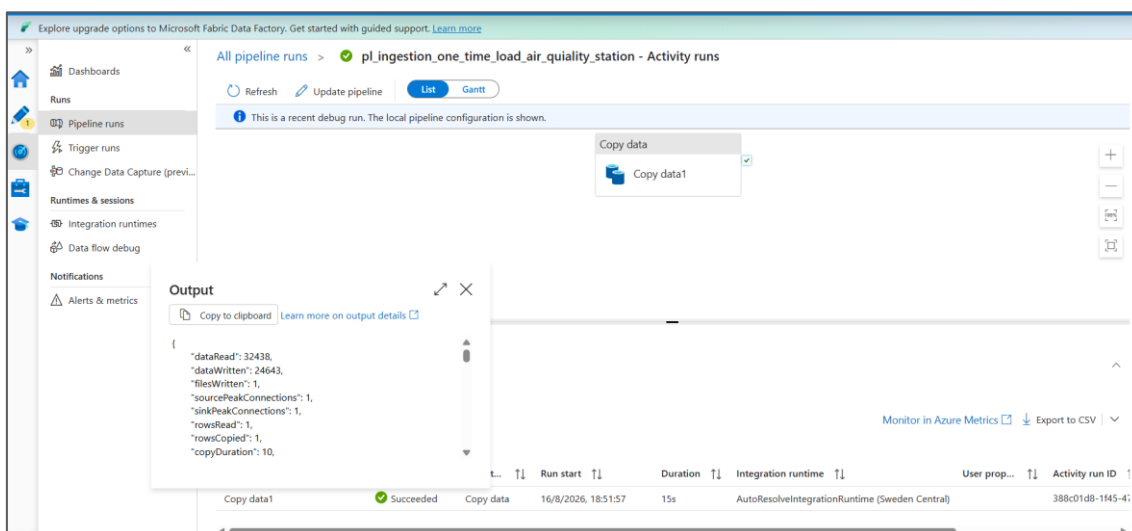
Lookup lookup_get_station

ForEach ForEach1

Monitor in Azure Metrics Export to CSV

	Run start	Duration	Integration runtime	User prop...	Activity run ID
data	15/8/2026, 22:56:44	18s	AutoResolveIntegrationRuntime (Sweden Central)		d106f287-e23e-
Copy data1	15/8/2026, 22:56:44	19s	AutoResolveIntegrationRuntime (Sweden Central)		19e56f62-d0cc
Copy data1	15/8/2026, 22:56:27	16s	AutoResolveIntegrationRuntime (Sweden Central)		05be995d-e45f
Copy data1	15/8/2026, 22:56:26	17s	AutoResolveIntegrationRuntime (Sweden Central)		64d331cc-067c
Copy data1	15/8/2026, 22:56:12	14s	AutoResolveIntegrationRuntime (Sweden Central)		93a499d2-65c0
Copy data1	15/8/2026, 22:56:11	13s	AutoResolveIntegrationRuntime (Sweden Central)		2d602ae2-9c8c
Copy data1	15/8/2026, 22:56:11	15s	AutoResolveIntegrationRuntime (Sweden Central)		2ad45553-591c
Copy data1	15/8/2026, 22:56:11	15s	AutoResolveIntegrationRuntime (Sweden Central)		c8c4e4ef-9b55
Copy data1	15/8/2026, 22:56:11	15s	AutoResolveIntegrationRuntime (Sweden Central)		733a377d-23f3

Anexo 7.3. Evidencia de ingesta de datos históricos de meteorología



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

Dashboard | Pipeline runs | Trigger runs | Change Data Capture | Runtimes & sessions | Integration runtime | Data flow debug | Notifications | Alerts & metrics

All pipeline runs > pl_ingestion_one_time_load_air_quality_station - Activity runs

Refresh Update pipeline List Gantt

This is a recent debug run. The local pipeline configuration is shown.

Copy data

Copy data1

Output

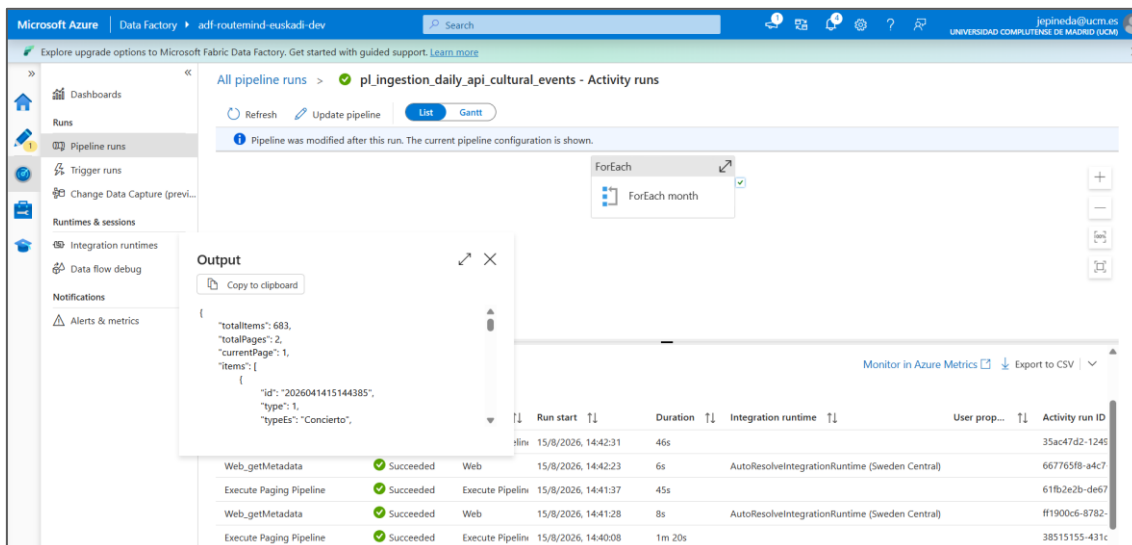
Copy to clipboard Learn more on output details

```

{
  "dataRead": 32438,
  "dataWritten": 24643,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "rowsRead": 1,
  "rowsCopied": 1,
  "copyDuration": 10,
}
    
```

Copy data1

Anexo 7.4. Evidencia de ingesta de datos Estaciones de calidad del aire



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

All pipeline runs > **pl_ingestion_daily_api_cultural_events - Activity runs**

Refresh Update pipeline List Gantt

Pipeline was modified after this run. The current pipeline configuration is shown.

ForEach

ForEach month

Output

Copy to clipboard

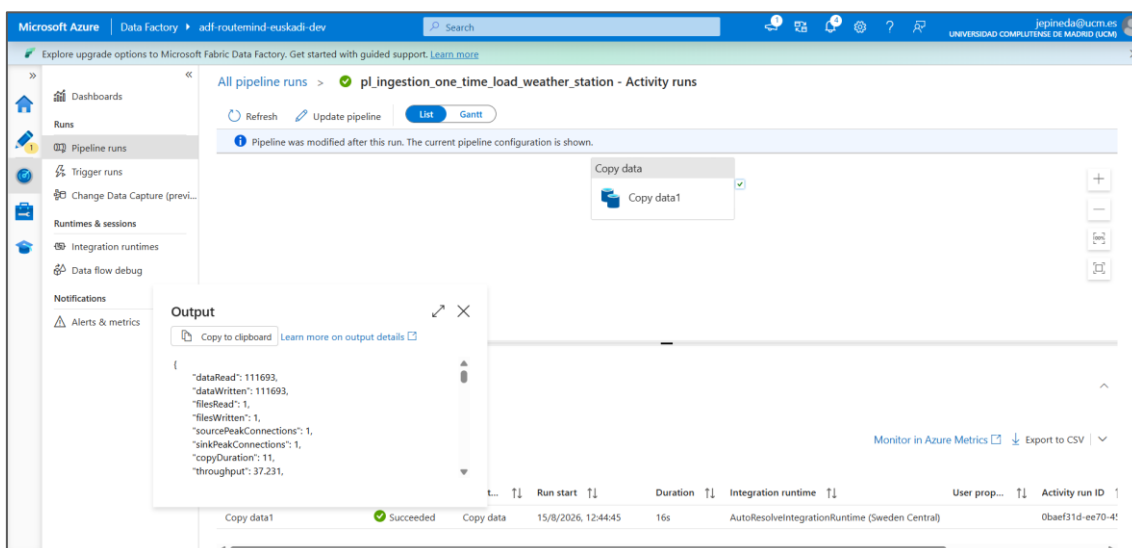
```

{
  "totalItems": 683,
  "totalPages": 2,
  "currentPage": 1,
  "items": [
    {
      "id": "2026041415144385",
      "type": 1,
      "typeEs": "Concierto",
    }
  ]
}
    
```

Monitor in Azure Metrics Export to CSV

Activity	Status	Type	Run start	Duration	Integration runtime	User prop...	Activity run ID
Web_getMetadata	Succeeded	Web	15/8/2026, 14:42:23	6s	AutoResolveIntegrationRuntime (Sweden Central)		35ac47d2-1245
Execute Paging Pipeline	Succeeded	Execute Pipeline	15/8/2026, 14:41:37	45s	AutoResolveIntegrationRuntime (Sweden Central)		61fb2e2b-de67
Web_getMetadata	Succeeded	Web	15/8/2026, 14:41:28	8s	AutoResolveIntegrationRuntime (Sweden Central)		#f1900c5-8782
Execute Paging Pipeline	Succeeded	Execute Pipeline	15/8/2026, 14:40:08	1m 20s	AutoResolveIntegrationRuntime (Sweden Central)		38515155-431c

Anexo 7.5. Evidencia de ingesta de datos de Eventos culturales



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

Explore upgrade options to Microsoft Fabric Data Factory. Get started with guided support. [Learn more](#)

All pipeline runs > **pl_ingestion_one_time_load_weather_station - Activity runs**

Refresh Update pipeline List Gantt

Pipeline was modified after this run. The current pipeline configuration is shown.

Copy data

Copy data1

Output

Copy to clipboard Learn more on output details

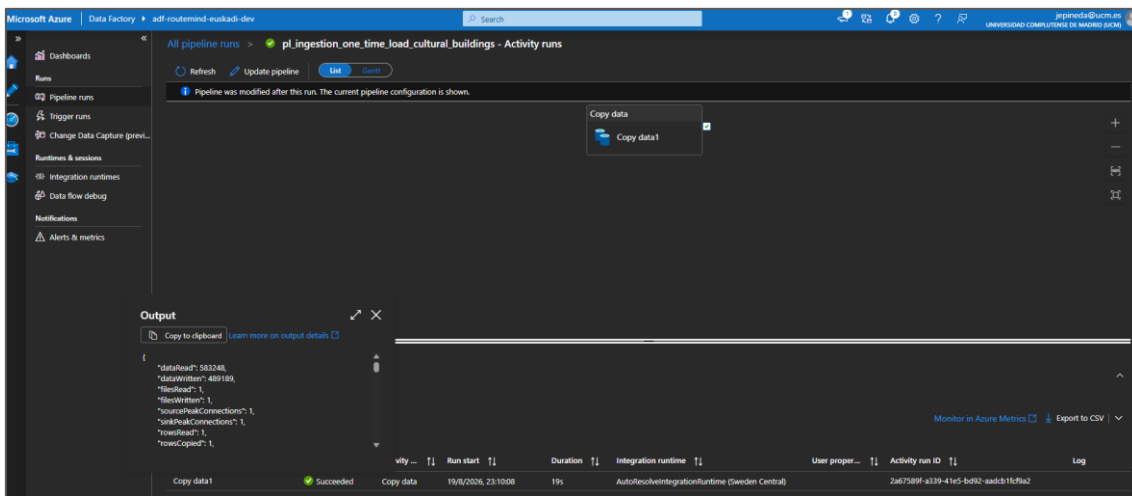
```

{
  "dataRead": 111693,
  "dataWritten": 111693,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "copyDuration": 11,
  "throughput": 37.231,
}
    
```

Monitor in Azure Metrics Export to CSV

Activity	Status	Type	Run start	Duration	Integration runtime	User prop...	Activity run ID
Copy data1	Succeeded	Copy data	15/8/2026, 12:44:45	16s	AutoResolveIntegrationRuntime (Sweden Central)		0baef31d-ee70-4

Anexo 7.6. Evidencia de ingesta de datos de Estaciones de meteorología



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

All pipeline runs > pl_ingestion_one_time_load_cultural_buildings - Activity runs

Refresh Update pipeline List Search

Pipeline was modified after this run. The current pipeline configuration is shown.

Copy data

Copy data1

Output

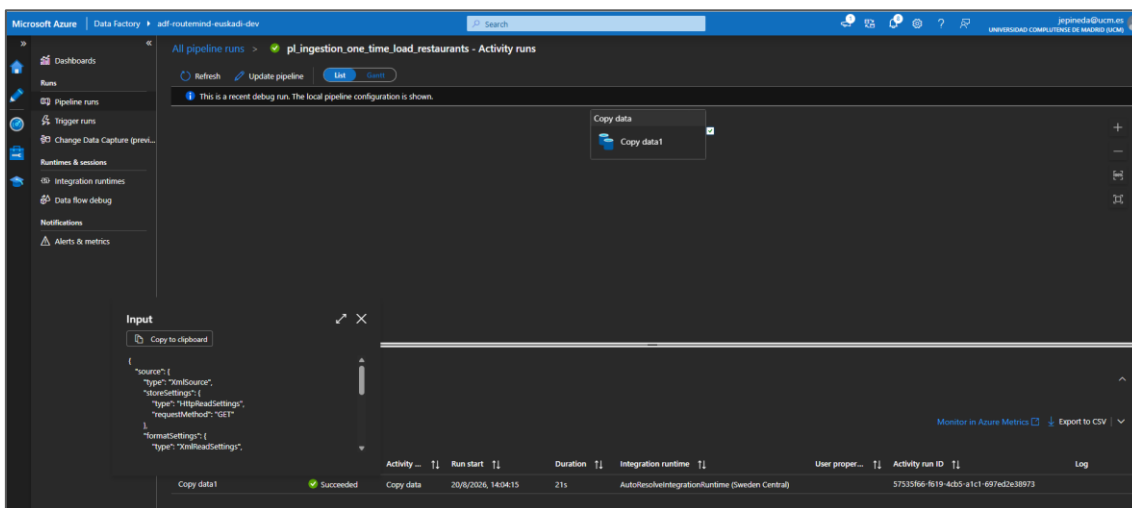
Copy to clipboard Learn more on output details

```
{
  "dataRead": 583248,
  "dataWritten": 489189,
  "fileRead": 1,
  "fileWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "timeRead": 1,
  "timeWrite": 1
}
```

Monitor in Azure Metrics Export to CSV

Activity	Run start	Duration	Integration runtime	User properties	Activity run ID	Log
Copy data1	15/6/2026, 23:10:08	19s	AutoResolveIntegrationRuntime (Sweden Central)		2a67589f-a139-41e5-bd92-aadcb1f6fa2	

Anexo 7.7. Evidencia de ingesta de datos de Edificios Culturales



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

All pipeline runs > pl_ingestion_one_time_load_restaurants - Activity runs

Refresh Update pipeline List Search

This is a recent debug run. The local pipeline configuration is shown.

Copy data

Copy data1

Input

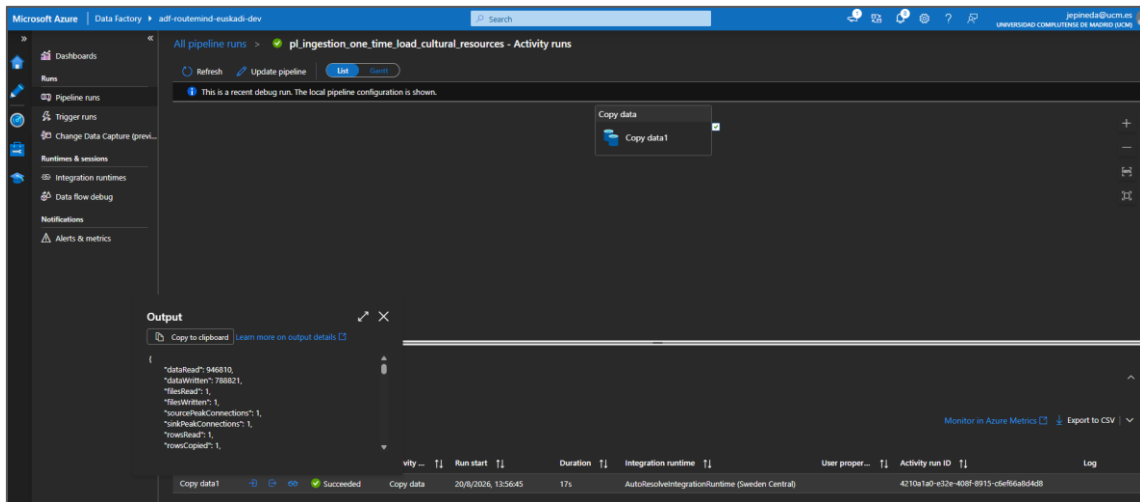
Copy to clipboard

```
{
  "source": {
    "type": "XmlSource",
    "storeSettings": {
      "type": "HttpReadSettings",
      "requestMethod": "GET"
    },
    "formatSettings": {
      "type": "XmlReadSettings"
    }
  }
}
```

Monitor in Azure Metrics Export to CSV

Activity	Run start	Duration	Integration runtime	User properties	Activity run ID	Log
Copy data1	20/6/2026, 14:04:15	21s	AutoResolveIntegrationRuntime (Sweden Central)		57332f66-8f19-4db5-81c1-697ed2e38973	

Anexo 7.8. Evidencia de ingesta de datos de Gastronomía



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

All pipeline runs > pl_ingestion_one_time_load_cultural_resources - Activity runs

Refresh Update pipeline List Queue

This is a recent debug run. The local pipeline configuration is shown.

Copy data

Copy data1

Output

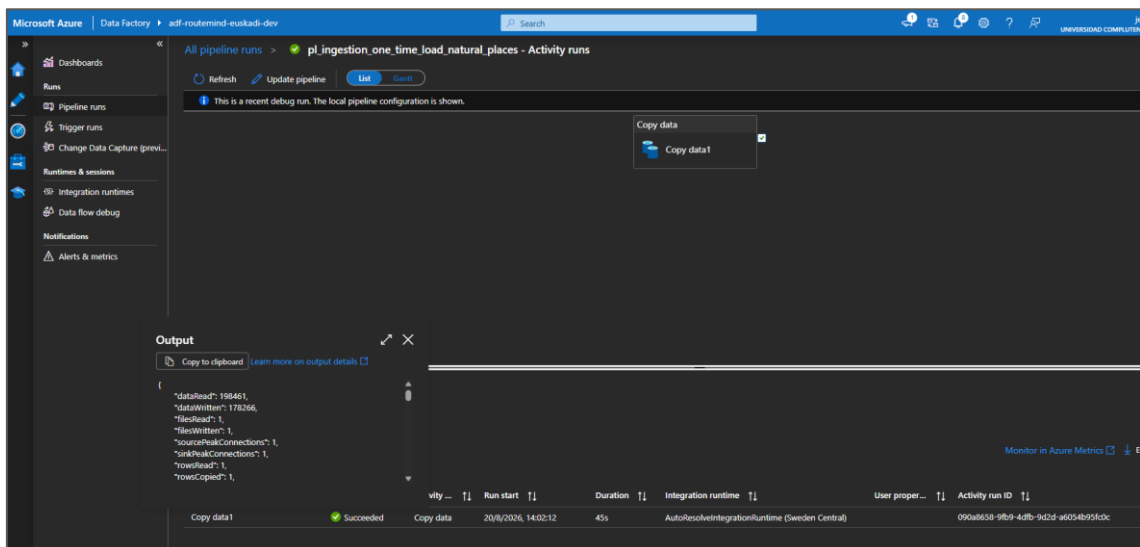
Copy to clipboard Learn more on output details

```
{
  "dataRead": 946810,
  "dataWritten": 788821,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "rowsRead": 1,
  "rowsCopied": 1
}
```

Monitor in Azure Metrics Export to CSV

Activity	Run start	Duration	Integration runtime	User proper...	Activity run ID	Log
Copy data1	20/8/2026, 13:56:45	17s	AutoResolveIntegrationRuntime (Sweden Central)		4210a1a0-e32e-408f-8915-c5e66a8d94b8	

Anexo 7.9. Evidencia de ingesta de datos de Recursos Culturales



Microsoft Azure | Data Factory | adf-routemind-euskadi-dev

All pipeline runs > pl_ingestion_one_time_load_natural_places - Activity runs

Refresh Update pipeline List Queue

This is a recent debug run. The local pipeline configuration is shown.

Copy data

Copy data1

Output

Copy to clipboard Learn more on output details

```
{
  "dataRead": 198461,
  "dataWritten": 178266,
  "filesRead": 1,
  "filesWritten": 1,
  "sourcePeakConnections": 1,
  "sinkPeakConnections": 1,
  "rowsRead": 1,
  "rowsCopied": 1
}
```

Monitor in Azure Metrics Export to CSV

Activity	Run start	Duration	Integration runtime	User proper...	Activity run ID	Log
Copy data1	20/8/2026, 14:02:12	45s	AutoResolveIntegrationRuntime (Sweden Central)		090a8658-98b9-4d8b-9d2d-a6054b95fcdc	

Anexo 7.10. Evidencia de ingesta de datos de Recursos Naturales

Catálogo Focus: ACTIVADO

cluster

Catalog

Escriba para buscar

Mi organización

dbw_routemind_euskadi_dev

bronze

Tablas (10)

- air_quality_history
- air_quality_stations
- cultural_buildings
- cultural_places
- events
- gastronomy
- municipality_codes
- natural_places
- weather_history
- weather_stations

default

gold

information_schema

silver

system

volumes

Explorador de catálogos > dbw_routemind_euskadi_dev >

bronze

Información general Detalles Permisos Políticas

Descripción

Generar con IA Añadir

Filtrar tablas

Tablas 10 Volúmenes 0 Modelos 0 Funciones 0

Ordenar

Nombre	Propietario	Creación
air_quality_history	jepineda@ucm.es	19 ago 2026, 22:51
air_quality_stations	jepineda@ucm.es	18 ago 2026, 23:23
cultural_buildings	jepineda@ucm.es	18 ago 2026, 20:35
cultural_places	jepineda@ucm.es	18 ago 2026, 22:34
events	jepineda@ucm.es	19 ago 2026, 20:15
gastronomy	jepineda@ucm.es	21 ago 2026, 14:07
municipality_codes	jepineda@ucm.es	04 sept 2026, 16:58
natural_places	jepineda@ucm.es	21 ago 2026, 14:06
weather_history	jepineda@ucm.es	24 ago 2026, 19:02
weather_stations	jepineda@ucm.es	19 ago 2026, 20:21

Anexo 8. Listado de tablas Bronze

Explorador de catálogos > dbw_routemind_euskadi_dev > bronze >

cultural_buildings

Información general Datos de muestra Detalles Permisos Políticas Historial Linaje Informaciones Calidad

Descripción

Generar con IA Añadir

Filtrar columnas...

Generar con IA

Columna	Tipo	Popularidad	Comentario	Etiquetas	Enmascaramiento de colum
rows	struct	all			
<pre> row: { "items": { "@num": "bigint", "address": "string", "country": "string", "countrycode": "bigint", "dataXML": "string", "documentDescription": "string", "documentName": "string", "friendlyUrl": "string", "latitudeLongitude": "string", "latwgs84": "double", "locality": "string", "lonwgs84": "double", "marks": { "metadataXML": "string", "municipality": "string", "municipalitycode": "string", "phone": "string", "physicalUrl": "string", "postalCode": "bigint", "qualityAssurance": "bigint", "qualityIconDescription": "string", "qualityIconURL": "string", "qualityQ": "bigint", "store": "bigint", "templateType": "string", "territory": "string", "territorycode": "string", "tourismEmail": "string", "visit": "bigint", "web": "string" } } } </pre>					
ingestion_timestamp	timestamp	all			
source_file	string	all			

Anexo 8.1. Ejemplo de tabla Bronze

Catálogo Focus: ACTIVADO

cluster

Catalog

Escriba para buscar

Mi organización

- dbw_routemind_euskadi_dev
 - bronze
 - default
 - gold
 - information_schema
 - silver**
 - Tablas (5)
 - air_quality_history_cleaned
 - events_cleaned
 - meteorology_stations
 - visit_points
 - weather_history_cleaned

Explorador de catálogos > dbw_routemind_euskadi_dev >

silver

Información general Detalles Permisos Políticas

Descripción

Generar con IA Añadir

Filtrar tablas

Ordenar

Nombre	Propietario	Creación
air_quality_history_cleaned	jepineda@ucm.es	30 ago 2026, 18:31
events_cleaned	jepineda@ucm.es	09 sept 2026, 00:18
meteorology_stations	jepineda@ucm.es	30 ago 2026, 18:16
visit_points	jepineda@ucm.es	23 ago 2026, 16:01
weather_history_cleaned	jepineda@ucm.es	30 ago 2026, 20:09

Anexo 9. Listado de tablas Silver

Explorador de catálogos > dbw_routemind_euskadi_dev > silver >

visit_points

Información general Datos de muestra Detalles Permisos Políticas Historial Linaje Informaciones Calidad

Descripción

Generar con IA Añadir

Filtrar columnas...

Generar con IA

Columna	Tipo	Popularidad	Comentario	Etiquetas	Enmascaramiento de colum
documentName	string				
templateType	string				
marks	string				
latwgs84	double				
lonwgs84	double				
municipalitycode	string				
territorycode	string				
category	string				

Anexo 9.1. Ejemplo de tabla Silver

Catálogo Focus: ACTIVADO

cluster

Catalog

Escriba para buscar

Mi organización

dbw_routemind_euskadi_dev

- bronze
- default
- gold**
 - Tablas (4)
 - events_user_category
 - visit_points_user_category
 - weather_features
 - weather_prediction_scoring
 - information_schema
 - silver

Explorador de catálogos > dbw_routemind_euskadi_dev >

gold

Información general Detalles Permisos Políticas

Descripción

Generar con IA Añadir

Filtrar tablas

Ordenar

Tablas 4 Volúmenes 0 Modelos 0 Funciones 0

Nombre	Propietario	Creación
events_user_category	jepineda@ucm.es	11 sept 2026, 14:44
visit_points_user_category	jepineda@ucm.es	09 sept 2026, 21:33
weather_features	jepineda@ucm.es	08 sept 2026, 14:41
weather_prediction_scoring	jepineda@ucm.es	08 sept 2026, 23:00

Anexo 10. Listado de tablas Gold

Explorador de catálogos > dbw_routemind_euskadi_dev > gold >

weather_features

Información general Datos de muestra Detalles Permisos Políticas

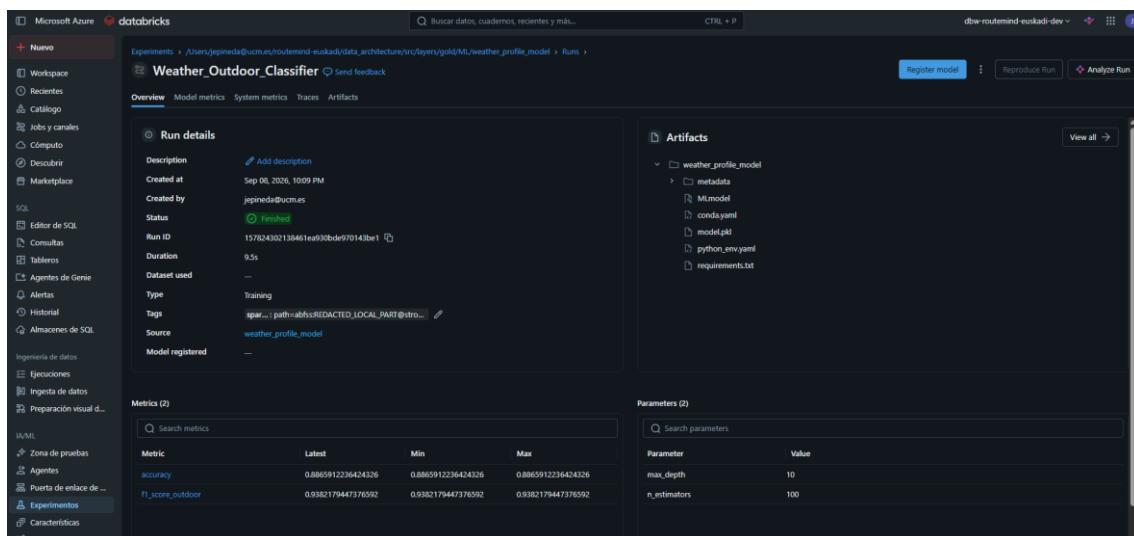
Descripción

Generar con IA Añadir

Filtrar columnas...

Columna	Tipo	Popularidad
date	date	...
municipalityCode	string	...
countyId	string	...
temp_max_mun	double	...
temp_min_mun	double	...
temp_avg_mun	double	...
precip_avg_mun	double	...
PM_10_avg_mun	double	...
PM2_5_avg_mun	double	...

Anexo 10.1. Ejemplo de tabla Gold



Weather_Outdoor_Classifier

Run details

Description: Add description

Created at: Sep 06, 2026, 10:09 PM

Created by: jepineda@ucm.es

Status: **Finished**

Run ID: 157624302130461ea930bde970143be1

Duration: 9.5s

Dataset used: -

Type: Training

Tag: spark...: path=abfs://REDACTED_LOCAL_PART@stro...

Source: weather_profile_model

Model registered: -

Artifacts

- weather_profile_model
 - metadata
 - MLmodel
 - conda.yaml
 - model.pkl
 - python_env.yaml
 - requirements.txt

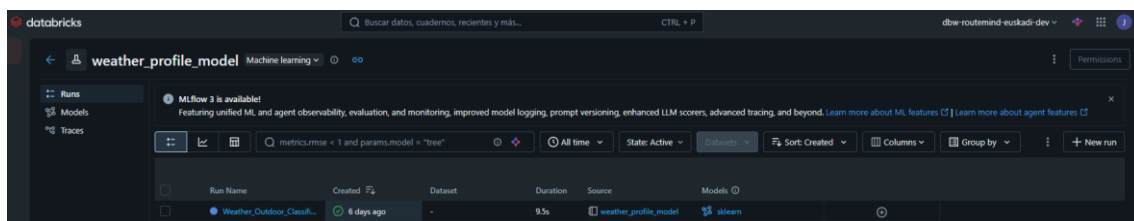
Metric (1)

Metric	Latest	Min	Max
accuracy	0.8865912236424326	0.8865912236424326	0.8865912236424326
f1_score_outdoor	0.9382179447376592	0.9382179447376592	0.9382179447376592

Parameters (2)

Parameter	Value
max_depth	10
n_estimators	100

Anexo 11. Evidencia del modelo de ML resultante con sus métricas.



weather_profile_model Machine learning

MLflow 3 is available

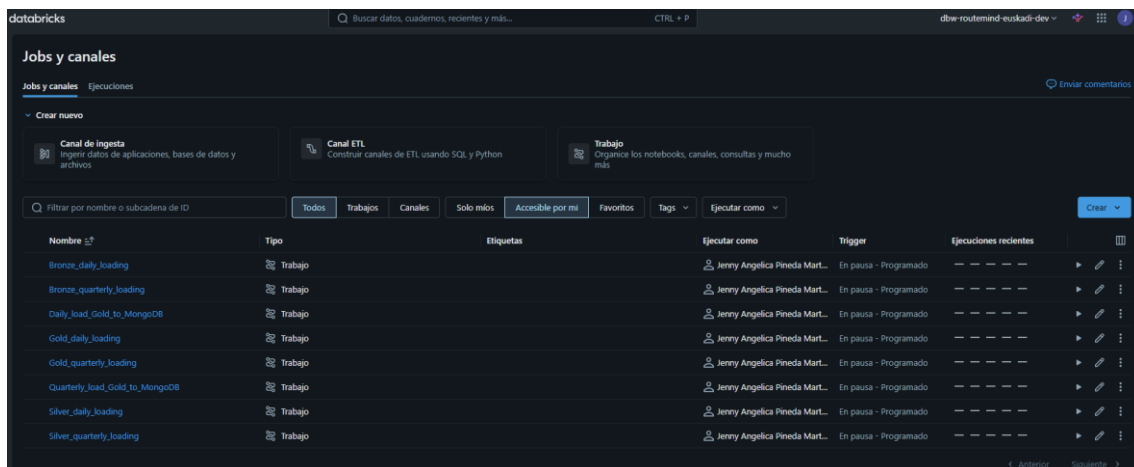
Featuring unified ML and agent observability, evaluation, and monitoring, improved model logging, prompt versioning, enhanced LLM scores, advanced tracing, and beyond. [Learn more about ML features](#) | [Learn more about agent features](#)

Search: metrics:loss < 1 and param:model = "tree"

Sort: Created

Run Name	Created	Dataset	Duration	Source	Models
Weather_Outdoor_Classifier	8 days ago	-	9.5s	weather_profile_model	sklearn

Anexo 11.1. Evidencia del modelo de ML resultante.



Jobs y canales

Jobs y canales Ejecuciones

Crear nuevo

Canal de ingesta: Ingesta datos de aplicaciones, bases de datos y archivos

Canal ETL: Construir canales de ETL usando SQL y Python

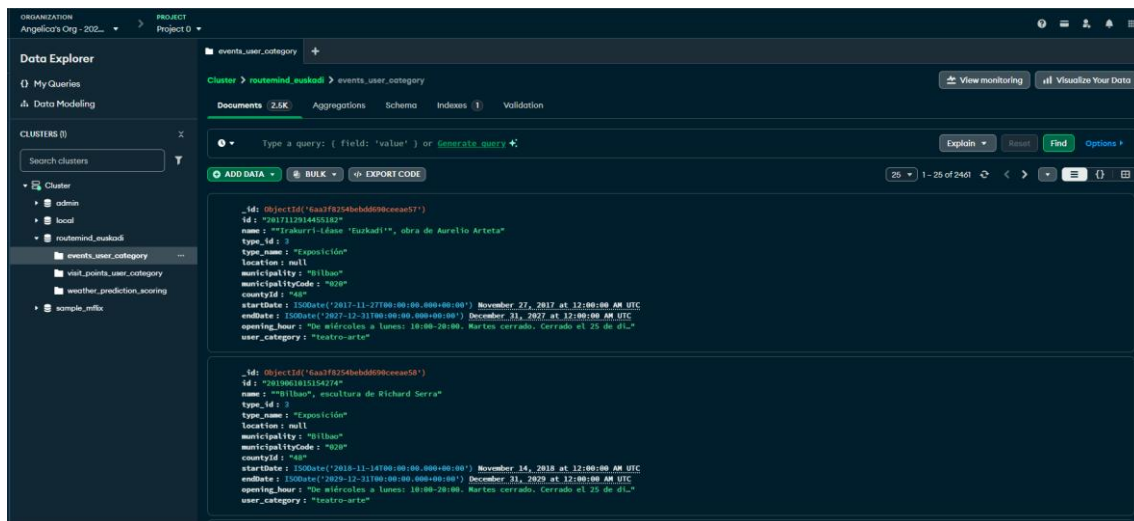
Trabajo: Organice los notebooks, canales, consultas y mucho más

Filtrar por nombre o subcadena de ID

Todos Trabajos Canales Solo míos Accesible por mí Favoritos Tags Ejecutar como

Nombre	Tipo	Etiquetas	Ejecutar como	Trigger	Ejecuciones recientes
Bronze_daily_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Bronze_quarterly_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Daily_load_Gold_to_MongoDB	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Gold_daily_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Gold_quarterly_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Quarterly_load_Gold_to_MongoDB	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Silver_daily_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶
Silver_quarterly_loading	Trabajo		Jenny Angelica Pineda Mart...	En pausa - Programado	-----▶

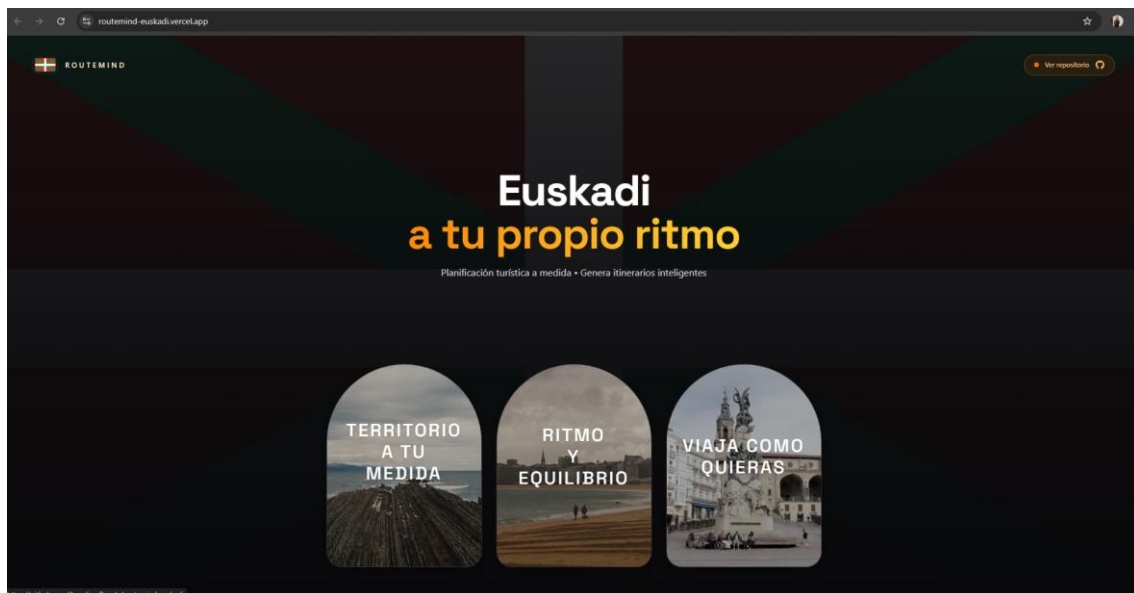
Ver Anexo 12. Jobs de Databricks con triggers asociados



Anexo 13. Evidencia de base de datos MongoDB con colecciones creadas

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
events_user_category	-	520.39 kb	826.67 kb	2.5K	335.00 B	1	139.26 kb
visit_points_user_category	-	286.72 kb	506.51 kb	1.8K	281.00 B	1	102.40 kb
weather_prediction_scoring	-	1.01 MB	2.35 MB	23K	104.00 B	1	983.04 kb

Anexo 13.1. Información de Colecciones de MongoDB



Anexo 14. Evidencia de aplicación Web RouteMind Euskadi.

PLANIFICADOR

Ajusta tu viaje route mind-euskadi-api 3 días

Rango de fechas: 17/09/2026 - 19/09/2026

Hora de llegada: 11:00

Hora de salida: 17:00

Transporte: Transporte Público

Ritmo: Equilibrado

Presupuesto: Alto

Personas: 2 Personas

Preferencia de planes: 3 planes elegidos

SITIOS CLAVE Bizkaia

Marca hasta tres lugares imprescindibles para ti. De esta manera Gemini estructurará un viaje armónico a su alrededor.

Museo Guggenheim Bilbao Casco Viejo de Bilbao Museo de Bellas Artes de Bilbao

Generar Itinerario →

Anexo 14.1. Formulario de preferencias de usuario en Web RouteMind Euskadi.

Itinerario generado con IA usando el modelo gemini-3.5-flash.

RESULTADO

Tu ruta planificada geminimongo

Escapada Gastronómica y Cultural en Bilbao

Itinerario de ritmo equilibrado ideado para 2 personas con transporte público y presupuesto alto, combinando alta gastronomía, tradición de sidrería y los iconos culturales indispensables de Bilbao.

17 DE SEPTIEMBRE DE 2026

Día 1: Vanguardia Artística y Gastronomía local

MAÑANA
Sin datos

MEDIODÍA
Almuerzo de bienvenida
Gastronomía vanguardista en la zona neurálgica de la ciudad nada más llegar.
Bistró Guggenheim Bilbao indoor

TARDE
Visita al Museo Guggenheim Bilbao
Ineludible referente arquitectónico y cultural seleccionado por el usuario.
guggenheim-bilbao indoor

NOCHE
Cena tradicional en Sidrería
Experiencia auténtica de sidrería en Bilbao para cumplir las preferencias del usuario.
Ander indoor

METEOROLOGÍA
Buen tiempo previsto para desplazamientos a pie.

TRANSPORTE
Conexión a pie y en tranvía por el centro urbano.

Comienzo del itinerario a las 11:00.

18 DE SEPTIEMBRE DE 2026

Día 2: Casco Antiguo y Alta Gastronomía

MAÑANA
Paseo histórico por el Casco Viejo

MEDIODÍA
Almuerzo clásico bilbaíno

TARDE
Sin datos

NOCHE
Cena de alta cocina

Anexo 14.1. Evidencia de itinerario generado en Web RouteMind Euskadi.

```
[.venv] PS C:\repos\repos_master\app_contenedores\routermind-euskadi\data_architecture> python -m pytest tests/test_cleaning_functions.py -v
===== test session starts =====
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\repos\repos_master\app_contenedores\routermind-euskadi\.venv\Scripts\python.exe
cachedir: .pytest.cache
rootdir: C:\repos\repos_master\app_contenedores\routermind-euskadi\data_architecture
collected 3 items

tests/test_cleaning_functions.py::test_explode_array_column_with_mock PASSED
tests/test_cleaning_functions.py::test_extract_and_cast_with_mock PASSED
tests/test_cleaning_functions.py::test_empty_string_to_null_with_mock PASSED

===== 3 passed in 4.57s =====
```

Anexo 15. Ejecución exitosa de test unitarios